МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Донецкий национальный технический университет»

КАФЕДРА «ПРОГРАММНАЯ ИНЖЕНЕРИЯ им. Л. П. ФЕЛЬДМАНА»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к выполнению лабораторных работ по дисциплине «Программирование систем с серверами баз данных»

для обучающихся по направлению подготовки
09.03.04 «Программная инженерия»,
направленность (профиль) «Инженерия программного обеспечения»
всех форм обучения

Донецк
2024

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Донецкий национальный технический университет»

КАФЕДРА «ПРОГРАММНАЯ ИНЖЕНЕРИЯ им. Л. П. ФЕЛЬДМАНА»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к выполнению лабораторных работ по дисциплине «Программирование систем с серверами баз данных»

для обучающихся по направлению подготовки
09.03.04 «Программная инженерия»,
направленность (профиль) «Инженерия программного обеспечения»
всех форм обучения

РАССМОТРЕНО
на заседании кафедры
«Программная инженерия
им. Л. П. Фельдмана»
Протокол №8 от 08.05.2024 г.

УТВЕРЖДЕНО
на заседании учебно-издательского
совета ДонНТУ
Протокол № 5 от 18.06.2024 г.

Донецк
2024

УДК 004.42:004.65(076)
М54

Составители:
Щедрин Сергей Валерьевич – старший преподаватель кафедры программной инженерии им. Л. П. Фельдмана ФГБОУ ВО «ДонНТУ»;

Бердюкова Светлана Сергеевна – ассистент кафедры программной инженерии им. Л. П. Фельдмана ФГБОУ ВО «ДонНТУ»;

Филипишин Дмитрий Александрович – ассистент кафедры программной инженерии им. Л. П. Фельдмана ФГБОУ ВО «ДонНТУ».

М54 **Методические указания к выполнению лабораторных работ по дисциплине «Программирование систем с серверами баз данных»** : для обучающихся по направлению подготовки 09.03.04 «Программная инженерия», направленность (профиль) «Инженерия программного обеспечения» /
ФГБОУ ВО «ДонНТУ», Каф. программной инженерии
им. Л. П. Фельдмана ; сост.: С. В. Щедрин, С. С. Бердюкова,
Д. А. Филипишин. – Донецк : ДонНТУ, 2024. – Систем. требования : Acrobat Reader. – Загл. с титул. экрана.

Методические указания разработаны с целью оказания помощи обучающимся, а также формированию устойчивых знаний и практических навыков по дисциплине «Программирование систем с серверами баз данных».

Ответственный за выпуск:
Зори Сергей Анатольевич – доктор технических наук, профессор, заведующий кафедрой программной инженерии им. Л. П. Фельдмана ФГБОУ ВО «ДонНТУ».

УДК 004.42:004.65(076)

ОБЩИЕ СВЕДЕНИЯ

Предлагаемый курс лабораторных работ по дисциплине «Программирование систем с серверами баз данных» имеет своей целью развитие у студентов знаний и умений в области теории и средств создания клиент-серверной информационной системы средствами СУБД PostgreSQL.

Порядок выполнения лабораторных работ:

– после формирования технического задания на выполнение курсовой и подписания его заведующим кафедрой, студент настраивает собственный сервер базы данных, после чего приступает к проектированию таблиц и интерфейса, а также формированию SQL-запросов;

– на занятии студент должен предъявить подготовленное задание, пояснить и обосновать правильность выполненной задачи преподавателю;

– к следующей лабораторной работе студент должен оформить отчет по выполненной работе, предварительно защитив его у преподавателя.

Подписанные преподавателем отчеты являются основной частью курсовой работы, которые впоследствии дополняются студентом до рабочего макета с целью максимального приближения к законченному самостоятельному программному продукту.

Составители:
Щедрин Сергей Валерьевич — старший преподаватель кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДонНТУ».
Бердюкова Светлана Сергеевна — ассистент кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДонНТУ».
Филипишин Дмитрий Александрович — ассистент кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДонНТУ».

Ответственный за выпуск:
Зори Сергей Анатольевич —доктор технических наук, профессор, заведующий кафедрой программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДонНТУ»,.

СОДЕРЖАНИЕ

ЛАБОРАТОРНАЯ РАБОТА 1 6
ЛАБОРАТОРНАЯ РАБОТА 2 11
ЛАБОРАТОРНАЯ РАБОТА 3 29
ЛАБОРАТОРНАЯ РАБОТА 4 30
ЛАБОРАТОРНАЯ РАБОТА 5 35
ЛАБОРАТОРНАЯ РАБОТА 6 36
ЛАБОРАТОРНАЯ РАБОТА 7 37
ЛАБОРАТОРНАЯ РАБОТА 8 53

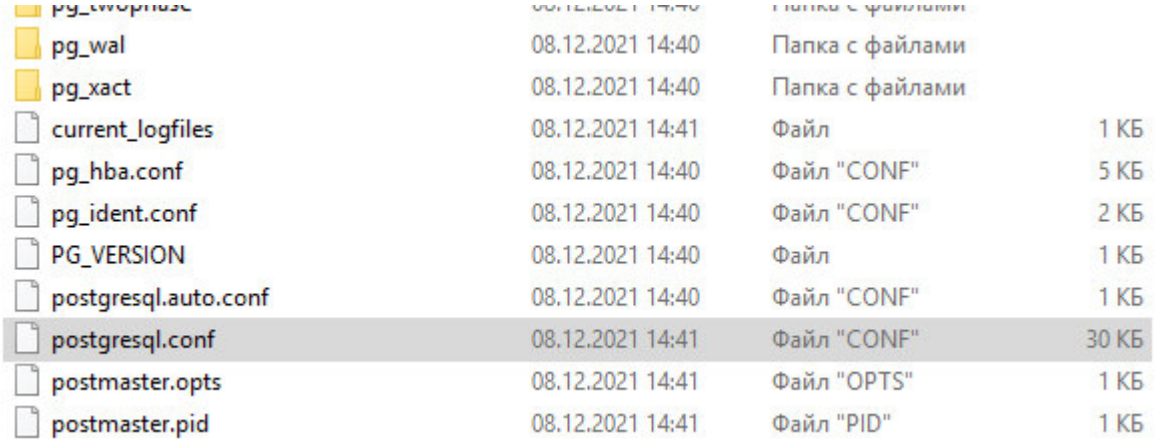
ЛАБОРАТОРНАЯ РАБОТА 1

Установка и настройка сервера

Цель работы: настройка и повышения производительности сервера на примере PostgreSQL.

Методические указания к выполнению работы.

Ознакомиться с возможными параметрами сервера влияющих на производительность СУБД. Эти параметры обычно устанавливаются в конфигурационном файле postgresql.conf, который расположен в папке data (пример пути C:\…\PostgreSQL\14\data), и влияют на все базы в текущей установке.



PostgreSQL не читает данные напрямую с диска и не пишет их сразу на диск. Данные загружаются в общий буфер сервера, находящийся в разделяемой памяти, серверные процессы читают и пишут блоки в этом буфере, а затем уже изменения сбрасываются на диск. Если процессу нужен доступ к таблице, то он сначала ищет нужные блоки в общем буфере. Если блоки присутствуют, то он может продолжать работу, если нет — делается системный вызов для их загрузки. Загружаться блоки могут как из файлового кэша ОС, так и с диска, и эта операция может оказаться весьма «дорогой».

Если объём буфера недостаточен для хранения часто используемых данных, то они будут постоянно писаться и читаться из кэша ОС или с диска, что крайне отрицательно скажется на производительности. В то же время не стоит устанавливать это значение слишком большим: это НЕ вся память, которая нужна для работы PostgreSQL, это только размер разделяемой между процессами PostgreSQL памяти, которая нужна для выполнения активных операций. Она должна занимать меньшую часть оперативной памяти вашего компьютера.

shared_buffers – это параметр отвечающий за общий буфер сервера.

```
#------------------------------------------------------------------------------
# RESOURCE USAGE (except WAL)
#------------------------------------------------------------------------------

# - Memory -

shared_buffers = 768MB          # min 128kB
```

Чтобы знать точное число shared_buffers, нужно учесть количество оперативной памяти компьютера, размер базы данных, число соединений и сложность запросов.

На выделенных серверах полезным объемом для shared_buffers будет значение 1/4 памяти в системе. Если у вас большие активные порции базы данных, сложные запросы, большое число одновременных соединений, длительные транзакции, вам доступен большой объем оперативной памяти или большее количество процессоров, то можно подымать это значение и мониторить результат, чтобы не привести к «деградации» (падению) производительности. Выделив слишком много памяти для базы данных, мы можем получить ухудшение производительности, поскольку PostgreSQL также использует кэш операционной системы.

Также следует помнить, что на 32 битной системе (Linux) каждый процесс лимитирован в 4 ГБ адресного пространства, где хотя бы 1 ГБ зарезервирован ядром. Это означает, что не зависимо, сколько на машине памяти, каждый PostgreSQL инстанс сможет обратиться максимум к 3 ГБ памяти. А значит максимум для shared_buffers в такой системе — 2–2.5 ГБ.

На Windows, большие значения для shared_buffers не столь эффективны, как на Linux системах, и в результате лучшие результаты можно будет получить, если установить это значение относительно небольшое (от 64 МБ до 512 МБ) и использовать кэш системы вместо него.

work_mem – память для сортировки результата запроса.

```
# Caution: it is not advisable to set max_prepared_transactions nonzero unless
# you actively intend to use prepared transactions.
work_mem = 128MB                                # min 64kB
```

Параметр определяет максимальное количество оперативной памяти, которое может выделить одна операция сортировки, агрегации и др. Это не разделяемая память, work_mem выделяется отдельно на каждую операцию (от одного до нескольких раз за один запрос).

Оптимальное значение параметра определяется следующим образом: количество доступной оперативной памяти (после того, как из общего объема вычли память, требуемую для других приложений, и shared_buffers) делится на максимальное число одновременных запросов, умноженное на среднее число операций в запросе, которые требуют памяти.

В качестве начального значения для параметра можете взять 2–4% доступной памяти. Для веб-приложений обычно устанавливают низкие значения work_mem, так как запросов обычно много, но они простые, обычно

хватает от 512 до 2048 КБ. С другой стороны, приложения для поддержки принятия решений с сотнями строк в каждом запросе и десятками миллионов столбцов в таблицах часто требуют work_mem порядка 500 МБ. Для баз данных, которые используются и так, и так, этот параметр можно устанавливать для каждого запроса индивидуально, используя настройки сессии.

Например, при памяти 1–4 ГБ рекомендуется устанавливать 32–128 MB.

max_connections – максимальное количество клиентов.

```
#------------------------------------------------------------------------------
# CONNECTIONS AND AUTHENTICATION
#------------------------------------------------------------------------------

# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                             # (change requires restart)
max_connections = 500                   # (change requires restart)
#superuser_reserved_connections = 3     # (change requires restart)
```

Параметр устанавливает максимальное количество клиентов, которые могут подключиться к PostgreSQL. Поскольку для каждого клиента требуется выделять память (work_mem), то этот параметр предполагает максимально возможное использование памяти для всех клиентов. Как правило, PostgreSQL может поддерживать несколько сотен подключений, но создание нового является дорогостоящей операцией.

huge_pages – поддержка больших страниц.

```
                                        # (change requires restart)
 huge_pages = try                       # on, off, or try
                                        # (change requires restart)
```

Когда объем памяти увеличивается на несколько десятков или сотни гигабайт, управлять ею становится сложнее, увеличиваются накладные расходы на адресацию памяти и поддержание страничных таблиц. Для облегчения жизни и были придуманы большие страницы, размер которых может быть от 1GB. За счет использования больших страниц можно получить ощутимый прирост скорости работы и увеличение отзывчивости в приложениях, которые активно работают с памятью.

Параметр huge_page может принимать три значения: off — не использовать большие страницы, on — использовать большие страницы, try — попытаться использовать большие страницы и в случае недоступности откатиться на использование обычных страниц. Значение try используется по умолчанию и является безопасным вариантом. В случае on, PostgreSQL не запустится, если большие страницы не определены в системе (или их недостаточно). После перезапуска базы с параметром try потребуется включить поддержку больших страниц в системе (по умолчанию они не задействованы).

temp_buffers — буфер под временные объекты, в основном для временных таблиц. Можно установить порядка 16 - 32 МБ.

max_prepared_transactions — количество одновременно подготавливаемых транзакций (PREPARE TRANSACTION). Можно оставить по умолчанию — 5.

max_stack_depth — специальный стек для сервера, который в идеале должен совпадать с размером стека, выставленном в ядре ОС. Установка большего значения, чем в ядре, может привести к ошибкам. Рекомендуется устанавливать 2–4 MB.

max_files_per_process — максимальное количество файлов, открываемых процессом и его подпроцессами в один момент времени. Рекомендуется уменьшить данный параметр, если в процессе работы наблюдается сообщение «Too many open files».

default_statistics_target — этот параметр задаёт объём статистики, собираемой командой ANALYZE. Увеличение параметра заставит эту команду работать дольше, но может позволить оптимизатору строить более быстрые планы, используя полученные дополнительные данные.

effective_cache_size — этот параметр сообщает PostgreSQL примерный объём файлового кэша операционной системы, оптимизатор использует эту оценку для построения плана запроса (указывает планировщику на размер самого большого объекта в базе данных, который теоретически может быть закеширован). На выделенном сервере имеет смысл выставлять effective_cache_size в 2/3 от всей оперативной памяти. На сервере с другими приложениями сначала нужно вычесть из всего объема RAM размер дискового кэша ОС и память, занятую остальными процессами.

random_page_cost — переменная, указывающая на условную стоимость индексного доступа к страницам данных. На серверах с быстрыми дисковыми массивами имеет смысл уменьшать изначальную настройку до 3.0, 2.5 или даже до 1.5. Если же активная часть вашей базы данных намного больше размеров оперативной памяти, попробуйте поднять значение параметра. Можно подойти к выбору оптимального значения и со стороны производительности запросов. Если планировщик запросов чаще, чем необходимо, предпочитает последовательные просмотры (sequential scans) просмотрам с использованием индекса (index scans), понижайте значение. И наоборот, если планировщик выбирает просмотр по медленному индексу, когда не должен этого делать, настройку имеет смысл увеличить. После изменения тщательно тестируйте результаты на максимально широком наборе запросов. Если вам кажется, что random_page_cost нужно еще понижать, разумнее в этом случае менять настройки статистики планировщика.

У PostgreSQL также есть специальная подсистема — сборщик статистики, которая в реальном времени собирает данные об активности сервера. Поскольку сбор статистики создает дополнительные накладные расходы на базу данных, то система может быть настроена как на сбор, так и не сбор статистики вообще.

track_counts — включать ли сбор статистики. По умолчанию включён, поскольку autovacuum требуется сбор статистики. Отключайте, только если статистика вас совершенно не интересует (как и autovacuum).

track_functions — отслеживание использования определенных

пользователем функций.

track_activities — передавать ли сборщику статистики информацию о текущей выполняемой команде и времени начала её выполнения. По умолчанию эта возможность включена. Следует отметить, что эта информация будет доступна только привилегированным пользователям и пользователям, от лица которых запущены команды, так что проблем с безопасностью быть не должно.

Задание

Установить PostgreSQL (или аналогичную СУБД) и произвести необходимые начальные настройки сервера для дальнейшей работы с СУБД.

Требования к выполняемой работе

- Задать значение параметру shared_buffers исходя из конфигурации компьютера и операционной системы, на которой установлен сервер;
- Задать значение параметру work_mem исходя из конфигурации компьютера, на которой установлен сервер;
- Задать значение параметру temp_buffers исходя из конфигурации компьютера, на которой установлен сервер;
- Задать значение параметру max_prepared_transactions исходя из конфигурации компьютера, на которой установлен сервер;
- Задать значение параметру max_files_per_process исходя из конфигурации компьютера, на которой установлен сервер;
- Задать параметру track_counts значение on

Требования к отчету по лабораторной работе

1. Титульный лист.
2. Тема лабораторной работы.
3. Задание на лабораторную работу.
4. Скриншоты выполненных требований.
5. Пояснения заданных значений для используемого сервера.

ЛАБОРАТОРНАЯ РАБОТА 2

Проектирование концептуальной модели базы данных

Цель работы: изучение вопросов представления данных об объектах предметной области и их связях в рамках реляционной модели.

Методические указания к выполнению работы.

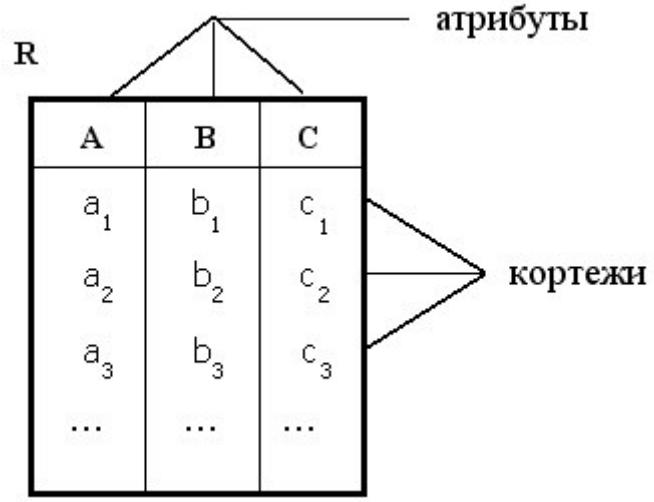
Основные определения

Каждая СУБД основывается на использовании определенной *модели данных.* **Модель данных** – это форма представления объектов предметной области и связей между ними.

В 1970 г. сотрудник корпорации IBM Э.Ф. Кодд, математик по образованию, использовал для обоснования модели данных математический аппарат теории множеств и предикативной логики. МД, предложенная Коддом, получила название *реляционной модели.* Ее идеи подробно рассмотрены в ставшей классической статье “Реляционная модель данных для больших совместно используемых банков данных”. Предложения Кодда были настолько эффективны для систем баз данных, что за эту модель он был удостоен престижной премии Тьюринга в области теоретических основ вычислительной техники.

Идеи Кодда оказали весьма существенное влияние на все аспекты технологии баз данных, а также на такие области, как искусственный интеллект, обработка естественных языков, разработка аппаратных систем.

Основной структурой данных в реляционной модели является *отношение*, именно поэтому модель получила название *реляционной* (от английского *relation* – отношение). Отношение имеет простую графическую интерпретацию: оно может быть представлено в виде *таблицы*:



R – отношение (таблица).

Кортеж - строка таблицы (включает данные об одном объекте (например, сведения о банке, о клиентах банка, анкетные данные студента и др.)).

Атрибут - столбец таблицы (столбцы представляют различные характеристики объекта (например, ФИО студента, номер его зачетной книжки, адрес и пр.)).

Множество значений, которые может принимать некоторый атрибут, называют *доменом.* Например, атрибут **Год рождения** имеет разные домены для таблицы **Студенты** и таблицы **Пенсионеры**.

Строки таблицы называют *записями.* Все записи имеют одинаковую структуру: они состоят из *полей*, в которых хранятся характеристики объекта. Каждое поле записи имеет строго определенный *тип данных.*

Определение реляционной модели данных (РМД): для хранения **данных** об объектах предметной области и для моделирования **взаимосвязей** между ними используется набор таблиц.

Таким образом, БД представляет собой множество взаимосвязанных таблиц, каждая из которых содержит информацию определенного типа.

Каждая таблица должна иметь *первичный ключ*: одно или несколько полей, которые имеют уникальные значения для каждой строки. Если первичный ключ достаточно сложный (состоит и з нескольких полей таблицы

или из одного, но длинного поля), то можно создать *искусственный первичный ключ*, представляющий собой некоторый *код*, для которого отводится одно поле. *Код* обычно является числовым или символьным данным. Использование *искусственного первичного ключа* гарантирует уникальность записей в таблице.

Представление связей между таблицами

Для моделирования связей между таблицами используется понятие *внешнего ключа. Внешний ключ — это поле таблицы, которое является первичным ключом в другой таблице; служит для организации связи между этими таблицами.* Для того чтобы реляционная БД была непротиворечивой, каждому значению внешнего ключа таблицы должно соответствовать значение первичного ключа связанной с ней таблицы. Это условие известно как *ссылочная целостность*. Создание связей между таблицами позволяет выбирать данные сразу из нескольких таблиц.

Между объектами предметной области возможны следующие виды связей:

– ”один-к-одному” (“1:1”);

– “один-ко-многим” (“1:n”);

– “многие-ко-многим” (“m:n”).

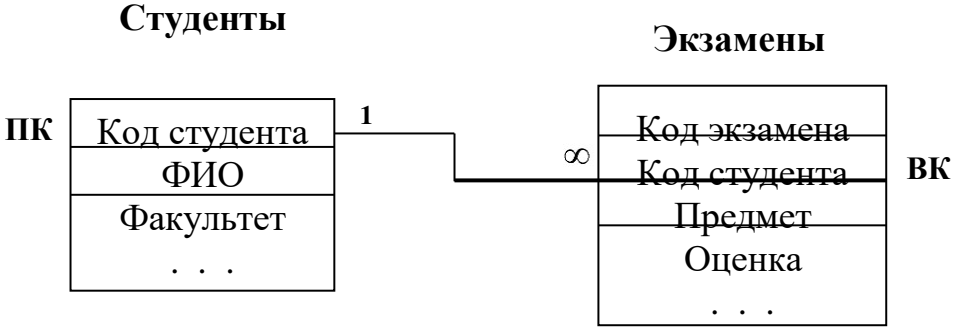
В большинстве случаев две таблицы связаны отношением **“1:n”**. Это означает, что одна запись первой таблицы (таблицы, в которой поле связи является первичным ключом; таблицы со стороны “один”) может быть связана с **несколькими** записями второй таблицы (таблицы, в которой поле связи является внешним ключом; таблицы со стороны “многие”). Однако, любая запись второй таблицы связана только с одной записью первой.

Для пояснения смысла связей между таблицами удобно использовать “родительскую” терминологию. Таблицу, в которой поле связи является первичным ключом, называют “отцом”, а таблицу, в которой поле связи является внешним ключом, - “сыном”. Связь “1:n” означает, что у “отца”

(таблица со стороны “один”) могут быть несколько “сыновей” (таблица со стороны “многие”). Другими словами: одна запись из таблицы – отца может быть связана с несколькими записями из таблицы – сына. Частные случаи этой связи: “отец” имеет одного “сына”, “отец” не имеет ни одного “сына”.

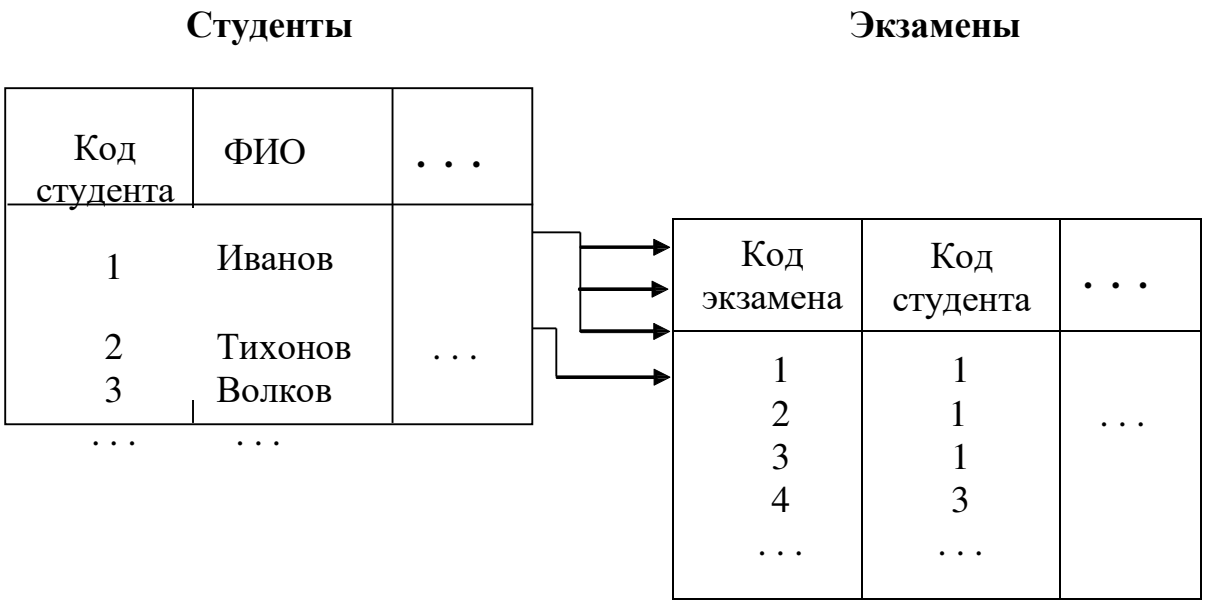
Ссылочную целостность, следуя данной терминологии, можно сформулировать так: не может быть “сына” без “отца”.

Пример: таблица **Студенты**, содержащая основные сведения о студентах, и таблица **Экзамены**, в которой содержатся сведения о результатах экзаменов, связаны отношением “1:n”:



Полем связи этих таблиц является поле **Код студента.**

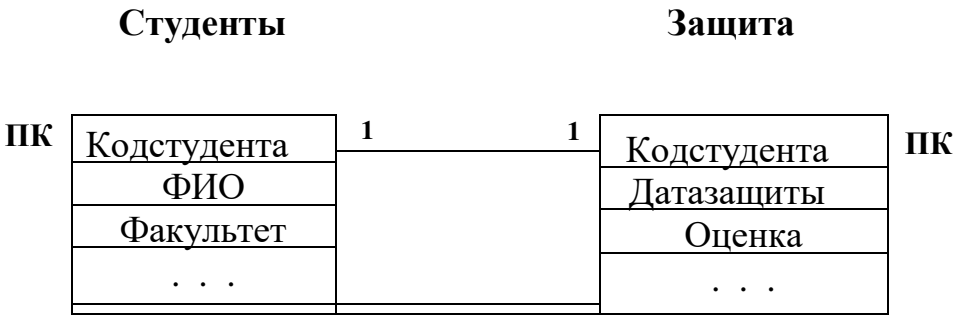
На уровне записей эта связь будет выглядеть так



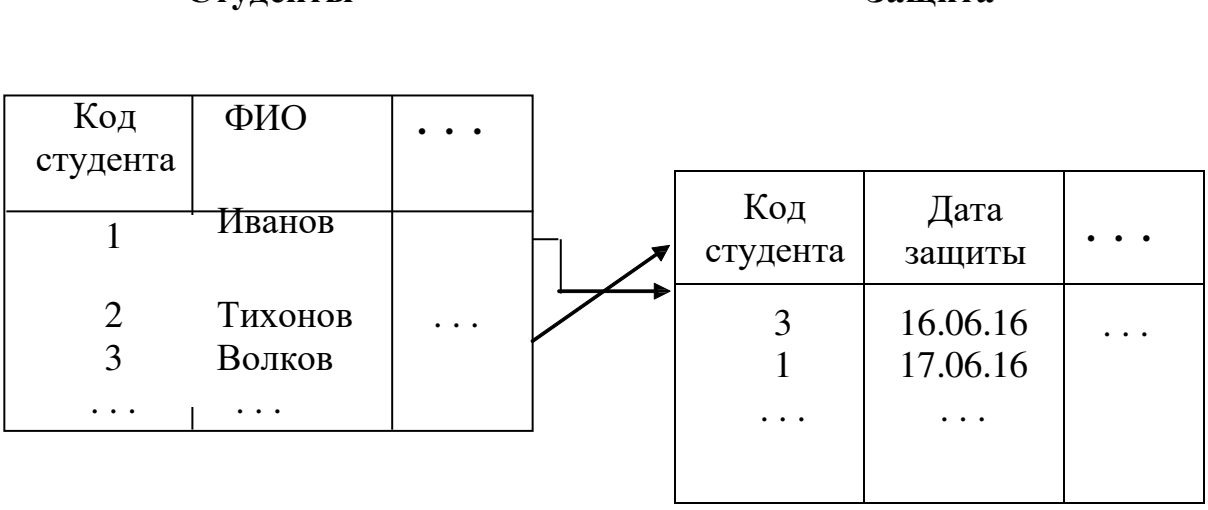
Студент Иванов (код **1**) сдал 3 экзамена (1:n), студент Волков (код **3**) сдал один экзамен (частный случай), студент Тихонов пока не сдал ни одного экзамена (частный случай).

Связь “1:1” означает, что любая запись первой таблицы связана только с одной записью второй таблицы (в частности, ни с одной). В этом случае обе таблицы должны иметь один и тот же первичный ключ.

Пример: таблица **Студенты**, содержащая основные сведения о студентах, и таблица **Защита**, в которой содержатся сведения о результатах защиты дипломных проектов, связаны отношением “1:1” (поле связи – **Код студента):**



На уровне записей связь выглядит так:



Студент Тихонов пока не защитил дипломный проект (частный случай).

Связь “m:n” означает, что одна запись первой таблицы может быть связана с несколькими записями второй таблицы и одна запись второй таблицы может быть связана с несколькими записями первой таблицы. Такой вид связи в РМД непосредственно не реализуется. Нужно создать дополнительную таблицу, называемую *таблицей пересечения*, с помощью которой одна связь

“m:n” будет сведена к двум связям “1:n” (таблица пересечения должна находиться со стороны “многие”).

Пример: таблица **Специальности,** содержащая сведения о специальностях института, и таблица **Дисциплины,** в которой приведен перечень читаемых в институте дисциплин, связаны отношением ”m:n” (с одной стороны, на одной специальности читается несколько дисциплин, с

Специальности

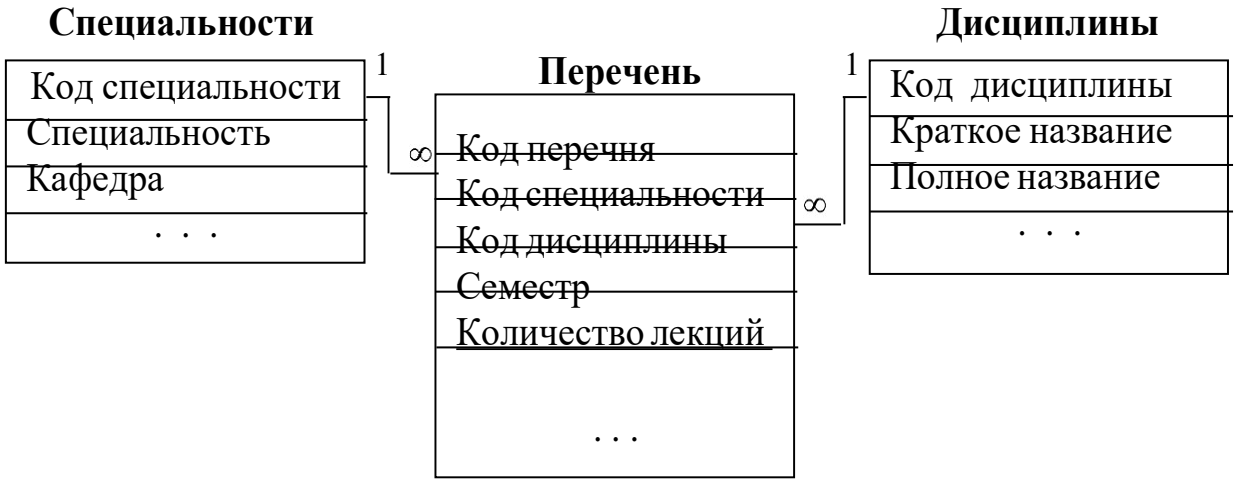
Код специальности
Специальность
Кафедра
. . .

Дисциплины

Код дисциплины
Краткое название
Полное название
. . .

другой стороны, некоторая дисциплина преподается на нескольких специальностях):

Нужно создать таблицу пересечения, в которой приводится перечень дисциплин по специальностям:



В таблице пересечения **Перечень** поля **Код специальности** и **Код дисциплины** являются внешними ключами.

Нормализация отношений

После анализа данных, характеризующих предметную область, отдельные данные объединяются в таблицы. Группировка данных в таблицы не имеет единственного решения. Можно получить несколько наборов таблиц, которые будут отображать одни и те же данные. Нужно определить оптимальный набор, который обладает лучшими свойствами при включении, удалении и модификации данных, а также позволяет сделать избыточность данных и время ответов на запросы к БД минимальными.

Основоположник РМД Э. Кодд с помощью разработанной им *теории нормализации* показал, какую структуру должны иметь таблицы, чтобы их набор был наиболее эффективным и, следовательно, был выбран в качестве концептуальной модели. Согласно теории нормализации таблица может находиться в одной из трех *нормальных форм*:

- *первая нормальная форма (*1НФ);
- *вторая нормальная форма* (2НФ),
- *третья нормальная форма* (3НФ).

Понятие нормальных форм связано с понятием *однозначной зависимости*: атрибут Y однозначно зависит от атрибута X, если каждому значению X соответствует только одно значение Y.

Каждая нормальная форма – это, прежде всего, *нормализованное отношение* (*нормализованная таблица).* Оно удовлетворяет следующим требованиям:

- *кортежи* в отношении (строки в таблице) должны быть различны;
- *порядок* кортежей (строк) не существенен;
- *порядок атрибутов (столбцов) не существенен;*
- *каждый* атрибут должен быть неделимым.

Таблицы, созданные в начале процесса проектирования концептуальной модели, обычно находятся в 1НФ.

1НФ – это нормализованное отношение, характеризующееся следующими недостатками:

- сложность в определении первичного ключа*;*
- избыточность данных;
- неоднозначные зависимости между неключевыми атрибутами и первичным ключом.

Для решения проблем, связанных с 1НФ, нужно привести таблицу ко второй нормальной форме.

2НФ: неключевые атрибуты однозначно зависят от первичного ключа, но, в то же время, однозначная зависимость может существовать и между неключевыми атрибутами отношения.

Для решения этой проблемы Кодд предложил для таблиц третью нормальную форму.

3НФ: отношение находится в третьей нормальной форме, если неключевые атрибуты его взаимно независимы и однозначно зависят только от первичного ключа.

Процесс преобразования таблиц из одной НФ в другую НФ называется *нормализацией*. Нормализация — это разбиение исходной таблицы на более мелкие таблицы с целью приведения их к 3НФ.

Таким образом, основную идею нормализации можно выразить так: **атрибуты отношения должны однозначно зависеть от первичного ключа, от всего ключа целиком и ни от чего другого, кроме ключа.**

Э. Кодд с помощью математического аппарата теории множеств доказал,

что только набор таблиц, находящихся в 3НФ, является оптимальным и может служить моделью реляционной базы данных (3НФ позволяет гораздо эффективнее выполнять операции включения, удаления и модификации записей БД, а также делает избыточность данных минимальной).

Выбор первичного ключа

Каждая таблица реляционной БД должна иметь *первичный ключ* (ПК): одно или несколько полей, однозначно определяющих каждую запись в таблице. Наличие в таблицах первичного ключа поддерживает целостность *данных*: СУБД не позволит ввести в это поле дублирующее значение или значение Null (пусто).

ПК может быть естественным или искусственным. Если поле содержит достаточно короткие уникальные значения, например, номера школ, больниц, почтовых отделений, то это поле можно определить как естественный первичный ключ.

В случае, когда уникальность записей не гарантируется значениями одного поля, нужно создать составной ключ. В случаях, когда имеем длинный естественный ПК, нужно создать искусственный первичный ключ – некий **код**, правила построения которого должны быть достаточно простыми.

Код может быть:

- числовым;
- символьным;
- счетчиком.

Числовой код (значения 1, 2, 3…) имеет смысл использовать в тех случаях, когда таблица содержит сравнительно немного записей. Он может занимать 1 байт (в таблице не может быть более 255 записей) или 2 байта (в таблице не более 32767 записей). Наращивание значений числового кода должно выполняться автоматически, без вмешательства пользователя. Для этого нужно написать макрос (модуль), в котором используются соответствующие системные функции.

Символьный код должен быть коротким: 1-2 символа. Например, можно взять первую букву длинного названия, если значения ее для записей таблицы будут уникальными (таблица НАЗВАНИЯ ЦЕХОВ примера 1). Однако, программная поддержка символьного кода (выделение букв из названий, проверка их на уникальность) достаточно сложна. Поэтому такой вид ПК может использоваться только для небольших сразу заполненных таблиц (таблиц-справочников).

Для таблиц, имеющих много записей и постоянно пополняющихся, самый удобный вид искусственного ПК – поле типа **Счетчик.** Значение такого поля автоматически увеличивается на единицу при добавлении новой записи в таблицу. Данный способ является самым легким для создания первичного ключа, но следует помнить, что в этом случае система гарантирует обеспечение уникальности только этого поля. Если же в таблице имеется другое поле, значения которого должны быть уникальными, то эту проблему должен решать программист.

Поле первичного ключа выполняет роль связующего поля между двумя таблицами. Кроме того, по умолчанию СУБД сортирует данные по первичному ключу, что значительно ускоряет выполнение запросов, а также поиск и сортировку данных.

Основой для всех современных СУБД является реляционная модель данных, базирующаяся на теории множеств и отношений (англ. *relation* – отношение). Отношение имеет простую графическую интерпретацию в виде таблицы. На логическом уровне БД представляет собой множество взаимосвязанных таблиц.

Каждая строка (*запись*, или *кортеж*) таблицы включает данные об одном объекте (например, банке, клиенте, студенте), а столбцы (*поля* или *атрибуты*) таблицы содержат различные характеристики этих объектов (например, наименование банка, фамилия и адрес клиента, номер зачетной книжки студента). Таким образом, *реляционной* называется модель данных, в которой для хранения данных об объектах предметной области и для моделирования взаимосвязей между ними используется набор таблиц.

После анализа предметной области отдельные данные объединяются в таблицы. Такая группировка не имеет единственного решения, поэтому нужно определить оптимальный набор, который обладает лучшими свойствами при включении, удалении и модификации данных, а также позволяет сделать избыточность данных и время ответов на запросы к БД минимальными.

Рассмотрим примеры проектирования концептуальной модели базы данных.

Пример 1

Для автоматизации учета работы молокозаводов области информационная система должна содержать такие данные о заводах: название завода, город, в котором находится завод, год начала функционирования, директор, телефон, цехи (название цеха, дата ввода в действие), выпускаемая продукция (тип продукции (сметана, кефир, ряженка, творог…), жирность, вес, дата выпуска, объем, цена за 1 кг.).

После первоначального анализа данных, характеризующих указанную предметную область, можно выделить объекты

ЗАВОДЫ, ПРОДУКЦИЯ.

Соответствующие им таблицы можно представить так:

ЗАВОДЫ

Завод	**Город**	**Начало**	**Дирек-тор**	**Тел**	**Цех**	**Дата ввода**
Геркулес	Донецк	1993	Иванов.	…	Творожный	…
Геркулес	Донецк	1993	Иванов	…	Жидких кисломолочных продуктов	
Геркулес	Донецк	1993	Иванов	…	Жидких пастеризованных продуктов	
Маричка	Горловка	1999	Петров	…	Творожный	
Маричка	Горловка	1999	Петров	…	Жидких кисломолочных продуктов	
Маричка	Горловка	1999	Петров	…	Жидких пастеризованных продуктов	
Лактис	Марьинка	1998	Власов	…	Творожный	
Лактис	Марьинка	1998	Власов	…	Жидких кисломолочных продуктов	
…	…	…	…	…	…	…

ПРОДУКЦИЯ

Тип	**Жирность**	**Вес**	**Дата выпуска**	**Цена единицы**
Молоко	…	?	?	?
Творог	…	?	?	?
Ряженка	…	?	?	?
……..	…			

Недостатки выполненного проектирования:

1. Избыточность данных (в таблице «Заводы» многократно повторяются данные о каждом заводе).
2. Неадекватность представления данных «Дата выпуска», «Вес» и «Цена единицы» в таблице «Продукция».
3. Отсутствие связи между таблицами.

Нормализуем таблицу «Заводы», разбив ее на 2 таблицы: «Заводы» (в ней будут приведены сведения о заводах; первичный ключ таблицы – поле «Завод») и «Цехи» (сведения о цехах заводов; поле «Завод» - внешний ключ; первичный ключ - составной: «Завод + Цех»).

ЗАВОДЫ

Завод	**Город**	**Начало**	**Директор**	**Телефон**
Геркулес	Донецк	1993	…	…
Маричка	Горловка	1999	…	…
Лактис	Марьинка	1998	…	…

ЦЕХИ

Завод	Цех	Дата ввода
Геркулес	Творожный	…
Геркулес	Жидких кисломолочных продуктов	…
Геркулес	Жидких пастеризованных продуктов	…
Маричка	Творожный	…
Маричка	Жидких кисломолочных продуктов	…
…..	…….	…

Между таблицами «Заводы» и «Цехи» существует связь “один-ко-многим” (один завод имеет несколько цехов). Так как названия цехов повторяются, то целесообразно организовывать справочник названий; в этой таблице поле «Код названия цеха» – искусственный ПК.

НАЗВАНИЯ ЦЕХОВ

Код названия цеха	Цех
Т	Творожный
К	Жидких кисломолочных продуктов
П	Жидких пастеризованных продуктов
….	……..

В таблице «Заводы» введем искусственный первичный ключ, чтобы в таблице «Цехи» заменить длинное поле «Завод» коротким полем «Код завода»:

ЗАВОДЫ

Код завода	Завод	Город	Начало	Директор	Телефон
1	Геркулес	Донецк	…	…	…
2	Маричка	Горловка	…	…	…
3	Лактис	Марьинка	…	…	…

Для таблицы «Цехи» составной ПК «Завод + Цех» также нужно заменить простым ключом. Для этого введем искусственный ПК «Код цеха»:

ЦЕХИ

Код цеха	**Код завода**	**Код названия цеха**	**Дата ввода**	**Количество**
1	1	Т	…	…
2	1	К	…	…
3	1	П Т	…	…
4	2	К	…	…
5	2	…	…	…
…	…		…	…

Между объектами ЦЕХИ–ПРОДУКЦИЯ существует отношение “m:n» (с одной стороны, каждый цех выпускает несколько видов продукции, с другой стороны, каждый вид продукции выпускается цехами нескольких заводов). Для представления этой связи необходимо ввести таблицу пересечения.

Разобьем таблицу «Продукция» на три: «Виды продукции», «Ассортимент» и «Выпускаемая продукция». «Код вида продукции», «Код продукции» – искусственные первичные ключи.

ВИДЫ ПРОДУКЦИИ

Код вида продукции	**Вид**
1	Сметана
2	Кефир
3	Ряженка …
…	

АССОРТИМЕНТ

Код продукции	**Код вида продукции**	**Жирность**
1	1	10
2	1	15
3	1	20
4	2	2,5
5	2	3,2
6	2	1

ВЫПУСКАЕМАЯ ПРОДУКЦИЯ

Код цеха	**Код продукции**	**Дата выпуска**	**Объем**	**Цена**
1	2	01.02.2016	120	17.90
1	3	01.02.2017	210	21.20
1	3	01.02.2016	220	26.40
5	2	02.02.2017	112	19.00

Таблица «Выпускаемая продукция» является таблицей пересечения («Код цеха» – внешний ключ, «Код продукции» - внешний ключ).

Естественный первичный ключ – составной: «Код цеха + Код продукции + Дата выпуска», поэтому введем короткий искусственный ПК.

Структура рассмотренных таблиц (все находятся в 3НФ) и связи между ними приведены на рис. 1.



Рисунок 1. Концептуальная модель данных для БД «Продукция молокозаводов»

Пример 2.

Для учета работы посреднической фирмы, занимающейся продажей вычислительной техники, необходимы сведения о поставщиках (название фирмы, город, страна, форма собственности, телефон, адрес) и о представленных ими товарах (наименование товара, фирма-изготовитель, дата поставки партии, цена единицы, количество единиц)

Первоначально рассмотрим представление БД в виде двух таблиц. Первая таблица – «Поставщики» - выглядит таким образом:

Фирма	**Город**	**Страна**	**Собственность**	**Телефон**	**Адрес**
Альфа	Москва	Россия	частная	...	Ул.Парковая, 334
Гамма	Минск	Беларусь	государственная	…	…
Омега	Киев	Украина	акционерная	…	…
…	…	…	…	…	…

Вторая таблица – «Товары»:

Товар	Изготовитель	Дата поставки	Цена единицы	Количество единиц
Принтер	Hewlett Packard	10.01.15	2000.00	10
DVD-RW	Acme	12.01.16	4.00	100
CD-RW	Memorex	12.02.16	2.00	300
CD-R	Acme	12.04.14	2.00	100
…	…	…	…	…

Данное представление имеет ряд недостатков:

1) Таблицы не связаны, следовательно, нельзя выяснить, кто поставлял заданный товар, какова общая стоимость товаров, представленных поставщиком, и т.п.

2) Избыточность в полях “Город”, “Страна”, “Собственность” – возможно многократное повторение одних и тех. Же заранее известных значений.

3) Присутствует аномалия обновления: если нужно изменить название товара, необходим просмотр всех его поставок.

4) Присутствует аномалия включения: нельзя включить в БД информацию о новом товаре до тех пор, пока не будет выполнена хотя бы одна его поставка/

5) Присутствует аномалия удаления: если некоторый товар исключен из ассортимента поставщиков, то нужно удалять записи обо всех его поставках.

6) Из-за того, что таблица «Поставщики» находится в НФ2 (присутствует зависимость между неключевыми полями «Страна» и «Город»), возможно занесение в таблицу заведомо недостоверных данных).

7) В реальной жизни между объектами «Поставщики» и «Товары» имеет место связь «многие ко многим»: один и тот же поставщик может поставлять разные товары, один и тот же товар может поставляться разными поставщиками. Для реализации этой связи введем таблицу пересечения, содержащую сведения о поставках товаров, поскольку явно такая связь представлена быть не может.

Для устранения избыточности в полях «Город», «Страна», «Собственность» введем справочники «Страны», «Города», «Собственность» с искусственными первичными ключами. Для связи включим одноименные внешние ключи в базовые таблицы.

Чтобы устранить аномалии обновления, включения и удаления, выделим отдельную таблицу, содержащую ассортимент товаров, предлагаемых поставщиками (таблица «Каталог»).

Чтобы избежать зависимости между неключевыми полями, информацию о городах и странах будем хранить в двух связанных таблицах.

Для отражения связи между объектами «Поставщики» и «Товары», присутствующей в реальной жизни, создадим таблицу пересечений «Товары», содержащую данные о дате, объеме поставки каждого товара и цене единицы.

Для реализации связей в каждую таблицу включим искусственный первичный ключ, а в подчиненные таблицы – внешний ключ.

Получим набор таблиц, показанный на рис. 2.

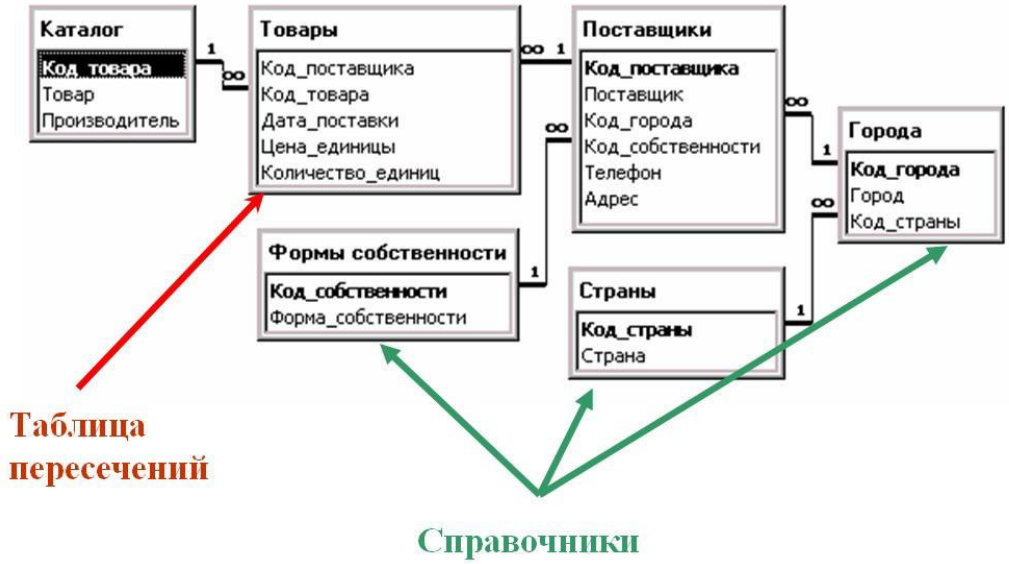


Рисунок 2 – Концептуальная модель данных для БД учета работы посреднической фирмы

Задание к лабораторной работе

Для предметной области, описанной в заданном варианте к курсовому проекту по курсу «Базы данных», выделить объекты и представить их взаимосвязанными таблицами, находящимися в третьей нормальной форме. Определить базовые таблицы и таблицы-справочники; указать для них первичные и внешние ключи. Описать типы полей.

Требования к отчету по лабораторной работе:

1. Титульный лист.
2. Тема лабораторной работы.
3. Задание на лабораторную работу.
4. Концептуальная модель к базе данных по варианту.
5. Указать первичные и внешние ключи.
6. Указать типы полей в основных таблицах и таблицах пересечения.

Контрольные вопросы

1. Сформулировать следующие определения:
   - предметная область, БД, СУБД;
   - реляционная модель данных;
   - нормализованная таблица, нормализация;
   - нормальные формы таблиц (первая, вторая, третья);
   - первичный, искусственный, внешний ключи;
   - целостность данных, ссылочная целостность.
2. Виды связей между объектами предметной области.

ЛАБОРАТОРНАЯ РАБОТА 3

Создание базы данных

Цель работы: освоение возможностей создания таблиц и установления связей между ними.

Методические указания к выполнению работы.

При создании базы данных необходимо придерживаться такой последовательности:

1. Создание таблиц базы данных, установка свойств полей и таблиц, создание подстановок для внешних ключей. При этом в первую очередь следует создавать главные таблицы, то есть находящиеся на линии связи со стороны «один», а после них – подчиненные;
2. Установление связей между таблицами, создание схемы данных;
3. Заполнение таблиц данными.

Задание к лабораторной работе

Для спроектированной реляционной модели в выбранной СУБД выполнить определение таблиц, указать типы и свойства полей (основные свойства, подстановка), создать схему данных.

Требования к отчету по лабораторной работе:

1. Титульный лист;
2. Тема лабораторной работы;
3. Задание на лабораторную работу;
4. Привести скриншот созданной схемы данных с указанными типами связей;
5. Привести 2-3 скриншота основных таблиц в режиме конструктора;
6. Привести 2-3 скриншота справочных таблиц в режиме конструктора.

ЛАБОРАТОРНАЯ РАБОТА 4

Проектирование интерфейса клиентской части курсового проекта

Цель работы: спроектировать интерфейс клиентской части курсового проекта.

Методические указания к выполнению работы.

На примере задания из курсового проекта разберем построение предполагаемого интерфейса для нескольких ролей.

Пример задания – абонент (ФИО или название предприятия, тип (частное лицо, школа, ВУЗ, предприятие,…), адрес, телефон, договор/договора (номер договора, информация о фирме-провайдере (название, тип собственности (государственный, частный, ЗАО, ОАО,…), адрес, телефон), тарифный план, дополнительные услуги (IPTV (кол-во каналов),тип IP-адреса (статический/динамический), IP- адрес, скидки (бизнес-время- 100% оплата, вечерний тариф- скидка 20%, ночной тариф- скидка 50%,…) …), скорость интернета, стоимость подключения, дата подключения)). Примечание: один абонент может заключать несколько договоров.

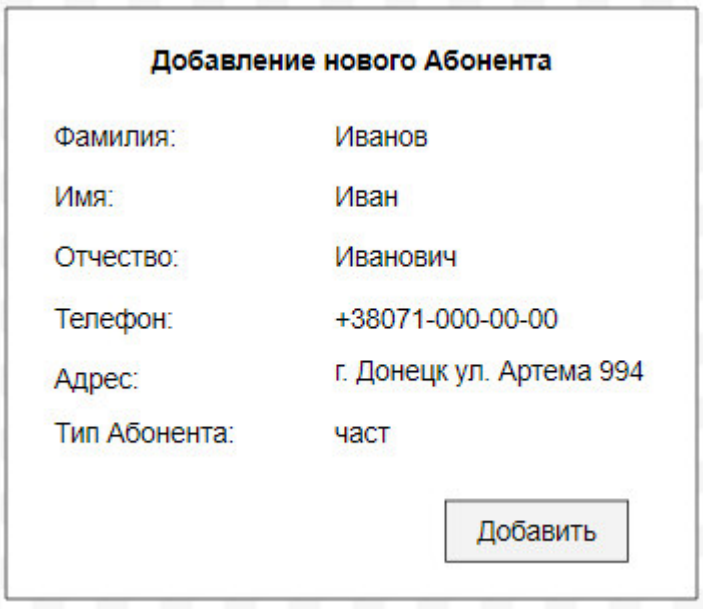
Допустим, будет 3 роли:

- **Оператор_1** который будет добавлять новых абонентов и их договора;
- **Оператор_2** который будет добавлять фирмы-провайдеры и информацию по ним;
- **Администратор**, который правит справочники и имеет возможность создавать новых операторов и назначать им необходимую роль.

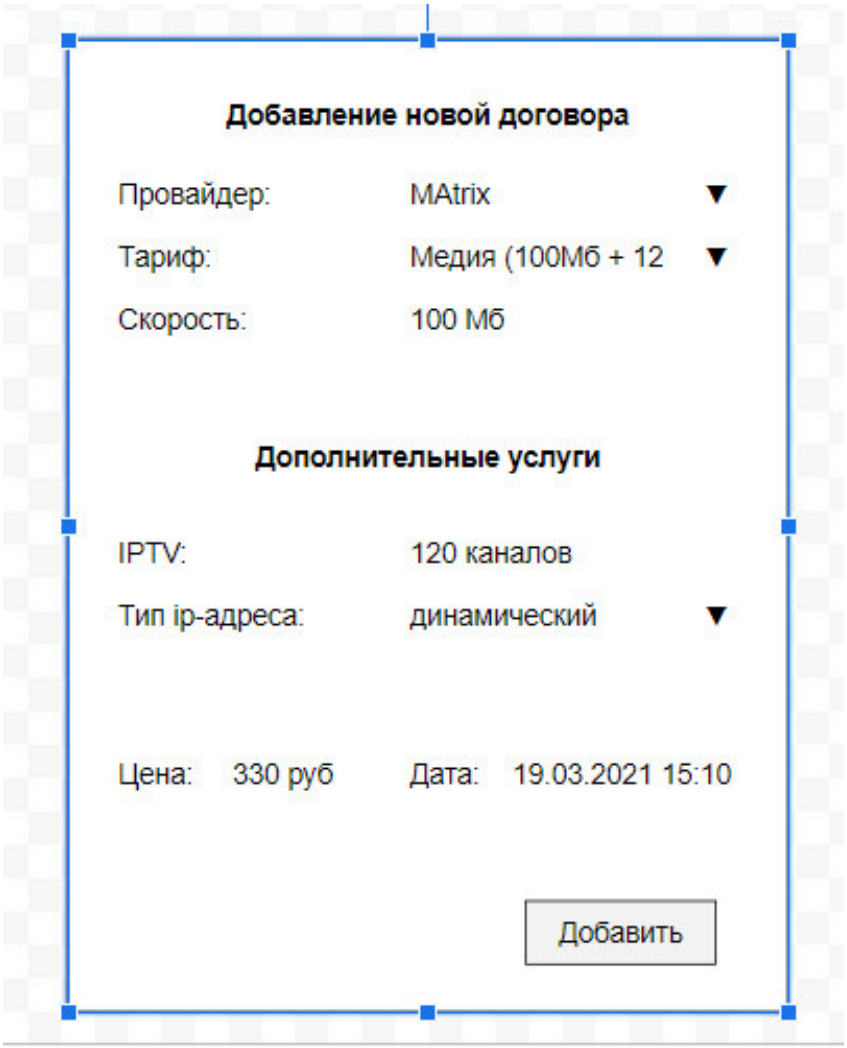
Пример интерфейса для роли Оператор_1:



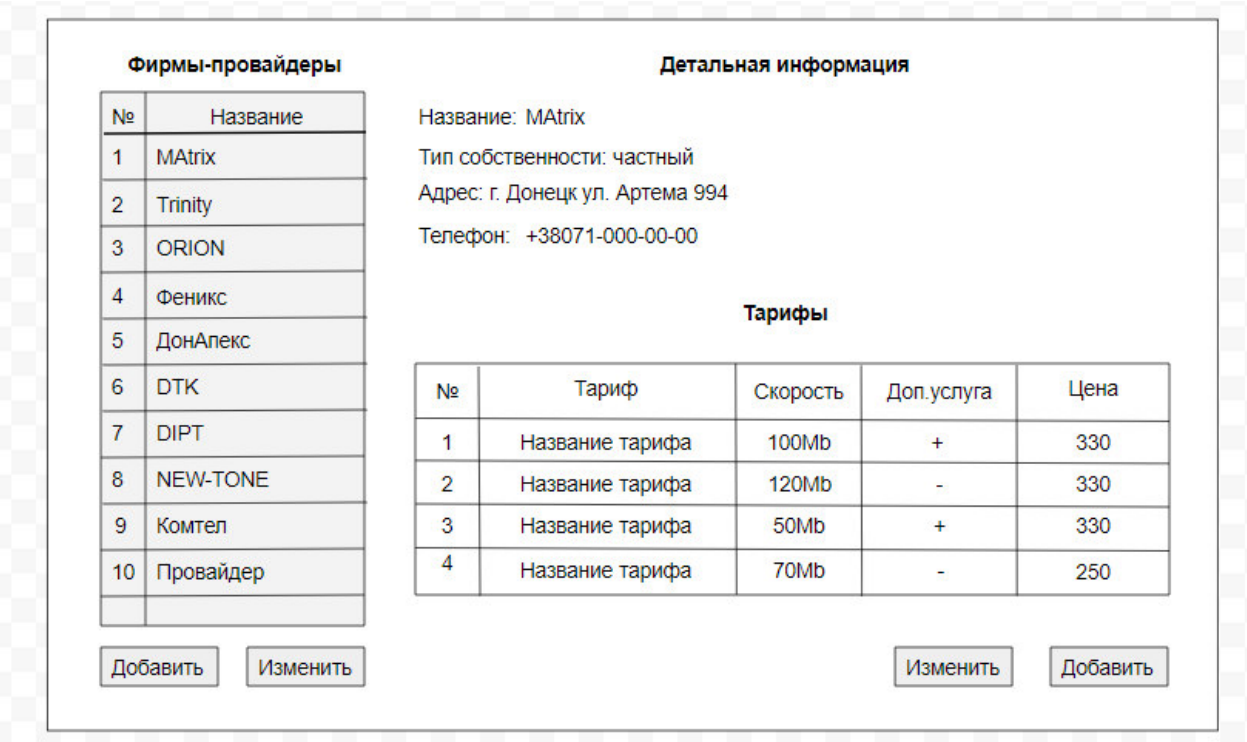
Добавление нового абонента/ редактирование абонента:



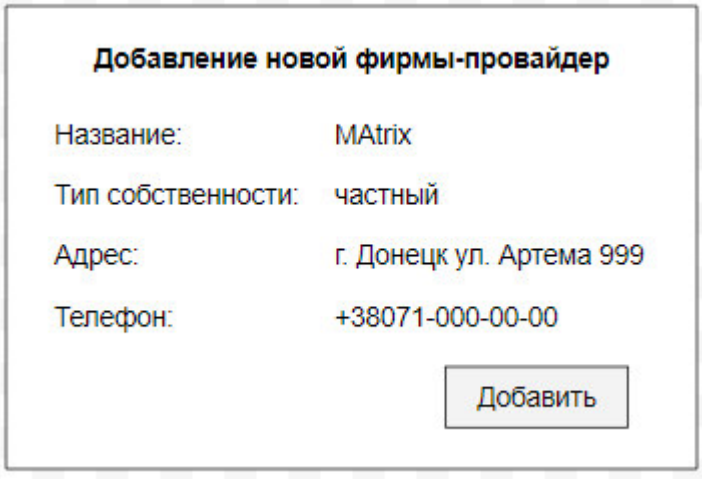
Добавление нового договора:



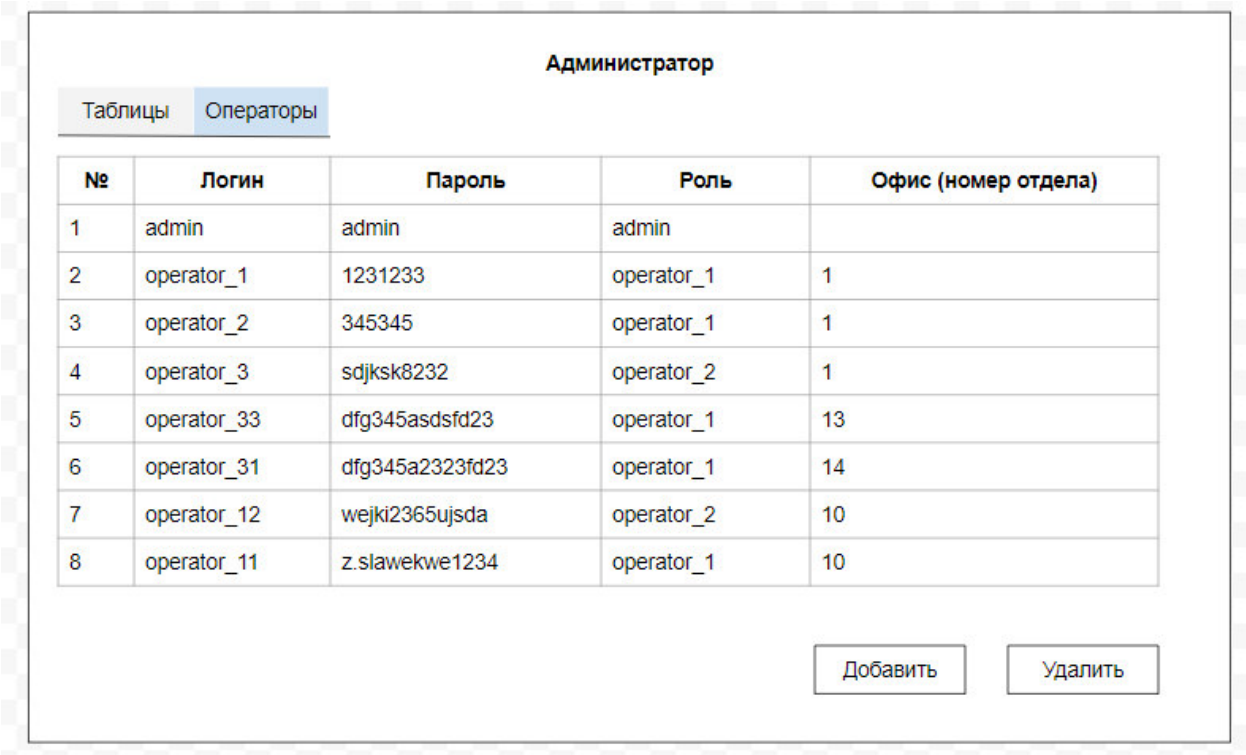
Пример интерфейса для роли Оператор_2:



Добавление нового фирмы-провайдера:



Пример интерфейса для роли Администратор:



Задание к лабораторной работе

Для предметной области, описанной в заданном варианте к курсовому проекту, в любой программе спроектировать интерфейс клиентской части курсового проекта для нескольких ролей (не менее 3-х). Согласовать с преподавателем проект интерфейса.

Требования к отчету по лабораторной работе:

1. Титульный лист.
2. Тема лабораторной работы.
3. Задание на лабораторную работу.
4. Скриншоты спроектированного интерфейса

ЛАБОРАТОРНАЯ РАБОТА 5

Разработка простых запросов на выборку

Цель работы: изучение способов получения информации с помощью запросов на выборку.

Методические указания к выполнению работы.

Запросы на выборку позволяют выбирать, сортировать, проводить вычисления с данными одной или нескольких взаимосвязанных таблиц.

Задание к лабораторной работе

1. Сформулировать следующие виды запросов на выборку:

– симметричное внутреннее соединение с условием (два запроса с условием отбора по внешнему ключу, два – по датам);

– симметричное внутреннее соединение без условия (три запроса);

– левое внешнее соединение;

– правое внешнее соединение;

– запрос на запросе по принципу левого ссоединения.

Запросы должны использовать данные из нескольких таблиц, условия отбора следует задавать через параметры;

2. Создать запросы, сформулированные в пункте 1;

3. Дополнить таблицы записями, согласующимися с разработанными запросами.

Требования к отчету по лабораторной работе:

1. Титульный лист;
2. Тема лабораторной работы;
3. Задание на лабораторную работу;
4. Сформулировать запросы к своей базе данных (см. вариант задания);
5. Привести код каждого из запросов по заданию;
6. Привести скриншоты результатов работы запросов.

ЛАБОРАТОРНАЯ РАБОТА 6

Разработка сложных запросов на выборку

Цель работы: изучение способов получения информации с помощью запросов на выборку.

Методические указания к выполнению работы.

Запросы на выборку позволяют выбирать, сортировать, проводить вычисления с данными одной или нескольких взаимосвязанных таблиц.

Задание к лабораторной работе

1. Сформулировать следующие виды запросов на выборку:
- итоговый запрос без условия;
- итоговый запрос с условием на данные;
- итоговый запрос с условием на группы;
- итоговый запрос с условием на данные и на группы;
- запрос на запросе по принципу итогового запроса;
- запрос с подзапросом.

Запросы должны использовать данные из нескольких таблиц, условия отбора следует задавать через параметры.

2. Создать запросы, сформулированные в пункте 1.
3. При необходимости дополнить таблицы записями, согласующимися с разработанными запросами.

Требования к отчету по лабораторной работе:

1. Титульный лист.
2. Тема лабораторной работы.
3. Задание на лабораторную работу.
4. Сформулировать запросы к своей базе данных (см. вариант задания).
5. Привести код каждого из запросов по заданию.
6. Привести скриншоты результатов работы запросов.

ЛАБОРАТОРНАЯ РАБОТА 7

Практическое применение документно-ориентированных баз данных

Цель работы: Изучение подходов построения и моделирования документно-ориентированных баз данных, с помощью типов данных JSON и JSONB на примере СУБД PostgreSQL, основываясь на принципах построения не реляционных (NoSQL) баз данных.

Методические указания к выполнению работы.

Разберем построение предполагаемой структуры JSON на примере задания из курсового проекта.

Пример задания – абонент (ФИО или название предприятия, тип (частное лицо, школа, ВУЗ, предприятие,…), адрес, телефон, договор/договора (номер договора, информация о фирме-провайдере (название, тип собственности (государственный, частный, ЗАО, ОАО,…), адрес, телефон), тарифный план, дополнительные услуги (IPTV (кол-во каналов),тип IP-адреса (статический/динамический), IP- адрес, скидки (бизнес-время- 100% оплата, вечерний тариф- скидка 20%, ночной тариф- скидка 50%,…) …), скорость интернета, стоимость подключения, дата подключения)). Примечание: один абонент может заключать несколько договоров.

Пример структуры:


```
{
  "name": "Petrov Ivan Petrovich",
  "type_abonent": "частное лицо",
  "phone": "+38071234567890",
  "address": "Artema 34",
  "dogovor": [
    {
      "firma": {
        "name": "Matrix",
        "phone": "+38071234567890",
        "address": "Vatutina 36",
        "type_sob": "частный"
      },
      "price": 294,
      "speed": 100,
      "tarif": "Медиа (100 Мбит + 120 IPTV)",
      "date_in": "2019-06-25 11:12:52.01897+03",
      "dop_usluga": {
        "name": "IPTV",
        "chanels": 120,
        "ip_address": "192.168.137.1",
        "type_ip_addres": "static"
      },
      "number_dogovora": 32766
    },
    {
      "firma": {
        "name": "Trinity",
        "phone": "+38071239387890",
        "address": "Popovicha 102",
        "type_sob": "государственный"
      },
      "price": 500,
      "speed": 120,
      "tarif": "120Mb",
      "type_addres": "dynamic",
      "number_dogovora": 301666
    }
  ]
}
```


Выборки из БД

Простые запросы:

```
SELECT data_jsonb ->>'name',
       data_jsonb ->>'phone',
       data_jsonb ->>'address',
       data_jsonb ->>'dogovor'
FROM json_table
WHERE data_jsonb ->> 'type_abonent' = 'предприятие';
```

name	phone	address	dogovor
Будур Захар Олегович	+38164802742762	ул. Ленина, 27	[{"data": "2020-10-16", "firma": {"name": "DIPT", "phone": "+38178117307161", "ad
Букалов Зураб Олесьевна	+38106771878894	ул. Ленина, 51	[{"data": "2020-08-28", "firma": {"name": "ORION", "phone": "+38131277632763", "
Бурзак Ирина Радионович	+38119338185172	ул. Куйбышева, д. 252	[{"data": "2020-06-24", "firma": {"name": null, "phone": "+38122954100448", "addre
Бурсов Карим Раимовна	+38145809468738	пр-т Ильича, д. 91	[{"data": "2020-12-14", "firma": {"name": "ORION", "phone": "+38106916910956", "
Бухало Лев Романович	+38100453076756	ул. Куйбышева, д. 252	[{"data": "2020-07-29", "firma": {"name": "Феникс", "phone": "+38157317472568", '
Буцик Лила Руслановна	+38158933286627	ул. Мушкетовская, д. 24	[{"data": "2020-11-18", "firma": {"name": "ORION", "phone": "+38200050117753", "
Валуев Мишель Тимурович	+38186471412967	ул. Ленина, д.15	[{"data": "2020-07-09", "firma": {"name": "ORION", "phone": "+38162397800895", "
Вальковец Моисей Турсунович	+38197218889829	ул. Ленина, 27	[{"data": "2020-07-18", "firma": {"name": "DIPT", "phone": "+38184846919340", "ad

```
SELECT data_jsonb ->>'name' as "name",
       data_jsonb #>>'{dogovor,1,firma,name}' as name_company,
       data_jsonb #>>'{dogovor,1,tarif}' as tarif,
       data_jsonb #>>'{dogovor,1,price}' as price
FROM json_table
WHERE data_jsonb->>'type_abonent'='юридическое лицо' AND
      jsonb_array_length(data_jsonb->'dogovor')>1
```

name	name_company	tarif	price
Черняк Рене Артуровна	DIDAN	Минимальный	947
Шевела Владена Ивановна	DIPT	Минимальный	867
Чудновский Алла Витальевич	EAST.NET	КОМФОРТ	316
Шерекин Дамир Максимовна	Trinity	Базовый	247
Чирка Алевтина Васильевич	ДонАпекс	Социальный	136
Шевцов Владислав Игоревна	DIPT	Минимальный	867
Шелудяков Георгий Леонтьевич	DTK	Коттедж-100	646

```
SELECT (jsonb_array_elements(data_jsonb->'dogovor'))->'firma'->>'name' as "name",
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'number_dogovora' as num,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'tarif' as tarif,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'data' as "data",
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'dop_usluga' as dop
FROM json_table
WHERE data_jsonb ->>'name'='Козаченко Зинаида Арифович';
```

name	num	tarif	data	dop
"DTK"	1506582051	"ПРЕМИУМ HD"	"2020-06-16"	(Null)
"Trinity"	1936926562	"Минимальный"	"2020-12-13"	(Null)
"NEW-TON"	1275614	"Расширенный HD"	"2020-12-01"	(Null)
"ДонАпекс"	324027997	"Мега HD (100 Мбит + 190 IPTV)"	"2020-08-26"	(Null)
"ДонАпекс"	2139548554	"Социальный"	"2021-02-25"	(Null)

Выборки с использованием jsonpath:

```
SELECT data_jsonb->>'name',
       data_jsonb->>'address',
       data_jsonb->>'phone'
FROM json_table
WHERE data_jsonb @@ '$.dogovor[*].firma.name == "Trinity"';
```

name	address	phone
Тяпкин Николь Валентинович	ул. Мушкетовская, д. 24	+38192275809060
Уваров Нина Валентиновна	ул. Ленина, д.15	+38149610129790
Удовиченко Олеся Валерьевич	пл. Победы, д. 27	+38180442053116
Утка Рене Владимировна	ул. Кавченко, 7	+38146565342218
Фероян Анжелика Жасурович	ул. Мушкетовская, д. 24	+38204288216225
Фидинчик Арина Ивановна	ул. Уральская, д. 17	+38128256385919

```
SELECT DISTINCT jsonb_path_query(data_jsonb, '$.dogovor[*].firma.name') as speed
FROM json_table
WHERE data_jsonb @@ 'exists($.dogovor[*] ? (@.speed < 100 && @.speed > 75)
      ? (@.price >= 999))';
```

speed
null
"DIDAN"
"EAST.NET"
"ORION"
"RAITCOM"
"TerraLine"
"ДонАпекс"
"Комтел"
"Феникс"

```
SELECT data_jsonb ->>'name',
       jsonb_path_query(data_jsonb,'$.dogovor.size()')
FROM json_table
WHERE jsonb_path_exists(data_jsonb,'$.dogovor[*] ? (@.firma.name=="DTK")
                                                 ? (@.tarif=="ПРЕМИУМ HD")');
```

?column?	jsonb_path_query
Береза Александр Аркадиевна	5
Даллакян Генрик Ромазанович	5
Даньшин Дамир Рушанович	5
Закутний Георгий Степанович	5
Иванников Людмила Богдановна	5
Иванченко Маргарита Бюльбюльевич	5
Иващенко Мирослава Валерьевна	5
Сухобоков Амина Егоровна	5
Тесленко Данила Ринатович	5

Выборки с использованием функций обработки JSON (не менее 3 штук)

```
SELECT *
FROM jsonb_populate_recordset(null::record,
      (SELECT jsonb_path_query_array(data_jsonb, '$.dogovor[*].firma')
       FROM json_table
       WHERE data_jsonb ->>'name'='Козаченко Зинаида Арифович')
      ) AS ("name" varchar, "phone" varchar, "address" text, "type_sob" text);
```

name	phone	address	type_sob
DTK	+38148795354823	ул. Гаврилова, д. 3	государственный
Trinity	+38157928386836	пл. Победы, д. 27	ЗАО
NEW-TON	+38204577709313	ул. Мушкетовская, д. 24	ЗАО
ДонАпекс	+38177082384096	ул. Мушкетовская, д. 24	ОАО
ДонАпекс	+38172665897113	ул. Шевченко, 9	ЗАО

```
SELECT *
FROM jsonb_populate_record(null::record,
  (SELECT jsonb_path_query(data_jsonb, '$.dogovor[*].firma?(@.name=="ДонАпекс")')
   FROM json_table
   WHERE data_jsonb ->>'name'='Фахрутдинов Светлана Геннадиевна')
  ) AS ("name" varchar, "phone" varchar, "address" text, "type_sob" text);
```

name	phone	address	type_sob
ДонАпекс	+38167854797636	ул. Ленина, д.15	государственный

```
SELECT jsonb_each_text(
            jsonb_path_query(data_jsonb,'$.dogovor[*].firma?(@.name=="DTK")'))
FROM json_table
WHERE jsonb_path_match(
            data_jsonb,'exists($.dogovor[*].firma.name ? (@ == "DTK"))')
      AND data_jsonb ->>'name'='Козаченко Зинаида Арифович'
```

jsonb_each_text
(name,DTK)
(phone,+38148795354823)
(address,"ул. Гаврилова, д. 3")
(type_sob,государственный)

Обновление/добавление данных:

```
UPDATE json_table
SET data_jsonb = jsonb_set(data_jsonb, '{dogovor,2,dop_usluga}',
                                       '{ "name": "IPTV",
                                          "chanels": 120,
                                          "ip_address": "102.168.183.1",
                                          "type_ip_addres": "static"}')
WHERE data_jsonb ->>'name'= 'Ришняк Дмитрий Максимовна' RETURNING *;
```

data_jsonb
{"name": "Ришняк Дмитрий Максимовна", "phone": "+38128236631130", "address": "ул. Ленина, 51", "dogovor": [{"data": "2021-01-

```
UPDATE json_table
SET data_jsonb = jsonb_insert(data_jsonb,
                        '{dogovor,2,dop_usluga,ip_address}','"192.168.137.1"')
WHERE jsonb_path_match(data_jsonb, 'exists($.dogovor[*].dop_usluga)')
AND data_jsonb ->> 'type_abonent'='юридическое лицо' RETURNING *;
```

id	data_jsonb
1096	{"name": "Мурас Василиса Григорьевна", "phone": "+38164055191292", "address": "ул. Советская, 79", "dogovor": [
1347	{"name": "Репин Дария Леонтьевич", "phone": "+38108011731582", "address": "просп. Семашко, 27а", "dogovor": [
27	{"name": "Алпеева Вильёта Васильевич", "phone": "+38107683233940", "address": "пр-т Ильича, д. 91", "dogovor":
3	{"name": "Абдуев Алексей Александрович", "phone": "+38181634913029", "address": "ул. Гаврилова, д. 3", "dogovor
9	{"name": "Авраменко Анатолий Андреевич", "phone": "+38188919195372", "address": "бул. Ильича, 8", "dogovor":
10	{"name": "Агаев Ангелина Андреевна", "phone": "+38129122781886", "address": "ул. Орджоникидзе, 155", "dogovo
17	{"name": "Акимов Артур Артемовна", "phone": "+38177464559285", "address": "пр. Металлургов, 35а", "dogovor":

```
UPDATE json_table
SET data_jsonb = data_jsonb -'dogovor'
WHERE data_jsonb ->>'name'='Рамазан Геннадий Ибрагимович' RETURNING data_jsonb;
```

data_jsonb
{"name": "Рамазан Геннадий Ибрагимович", "phone": "+38148704782371", "address": null, "type_abonent": "предприятие"}

```
UPDATE json_table
SET data_jsonb = data_jsonb || '{"address":"Новослободская 496"}'
WHERE data_jsonb ->>'name'='Рамазан Геннадий Ибрагимович' RETURNING data_jsonb;
```

data_jsonb
{"name": "Рамазан Геннадий Ибрагимович", "phone": "+38148704782371", "address": "Новослободская 496", "type_abonent": "пр

Удаление:

```
DELETE FROM json_table WHERE data_jsonb ->>'name'='Цыбенко Каролина Усама';
```

```
DELETE FROM json_table WHERE data_jsonb ->>'name'='Цыбенко Каролина Усама'
> Affected rows: 1
> Time: 0,017s
```

```
DELETE
FROM json_table
WHERE jsonb_path_match( data_jsonb,
                        'exists($.dogovor[*]?(@.type_addres=="dynamic"))');
```

```
DELETE FROM json_table
       WHERE jsonb_path_match(data_jsonb, 'exists($.dogovor[*]? (@.type_addres == "dynamic"))')
> Affected rows: 1
> Time: 0,019s
```

Создание индексов:

```
CREATE INDEX json_table_data_jsonb_idx ON json_table
USING GIN (data_jsonb);
CREATE INDEX json_table_data_jsonb_dogovor_idx ON json_table
USING GIN ((data_jsonb-> 'dogovor'));
CREATE INDEX json_table_data_jsonb_name_idx ON json_table
USING GIN ((data_jsonb-> 'name'));
```

Анализ и оптимизация запросов:

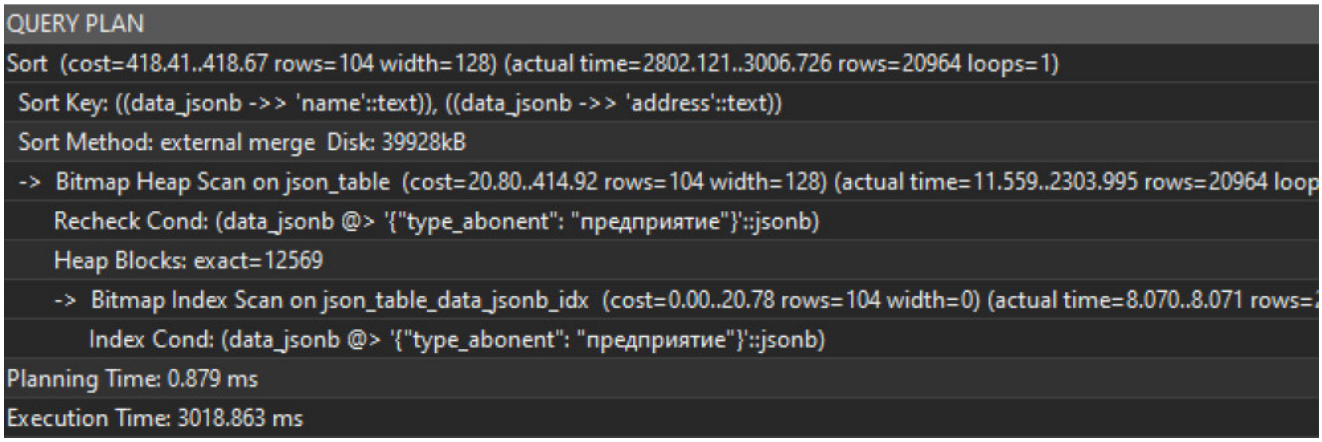
Исходный запрос:

```
EXPLAIN (ANALYZE)
   SELECT data_jsonb ->>'name' as "name",
                data_jsonb ->>'phone' as phone,
                data_jsonb ->>'address' as address,
                data_jsonb ->>'dogovor' as dogovor
   FROM json_table
   WHERE data_jsonb ->> 'type_abonent' = 'предприятие' ORDER BY "name", address;
```

```
QUERY PLAN
Sort  (cost=18841.57..18842.87 rows=518 width=128) (actual time=4199.246..4390.099 rows=20964 loops=1)
  Sort Key: ((data_jsonb ->> 'name'::text)), ((data_jsonb ->> 'address'::text))
  Sort Method: external merge  Disk: 39912kB
  ->  Seq Scan on json_table  (cost=0.00..18818.22 rows=518 width=128) (actual time=0.458..3686.145 rows=20964 loops=1)
        Filter: ((data_jsonb ->> 'type_abonent'::text) = 'предприятие'::text)
        Rows Removed by Filter: 82572
Planning Time: 0.307 ms
Execution Time: 4401.948 ms
```

Оптимизированный запрос:

```
EXPLAIN (ANALYZE)
   SELECT data_jsonb ->>'name' as "name",
          data_jsonb ->>'phone' as phone,
          data_jsonb ->>'address' as address,
          data_jsonb ->>'dogovor' as dogovor
   FROM json_table
   WHERE data_jsonb @> '{"type_abonent":"предприятие"}' ORDER BY "name", address;
```



По результатам видим, что:

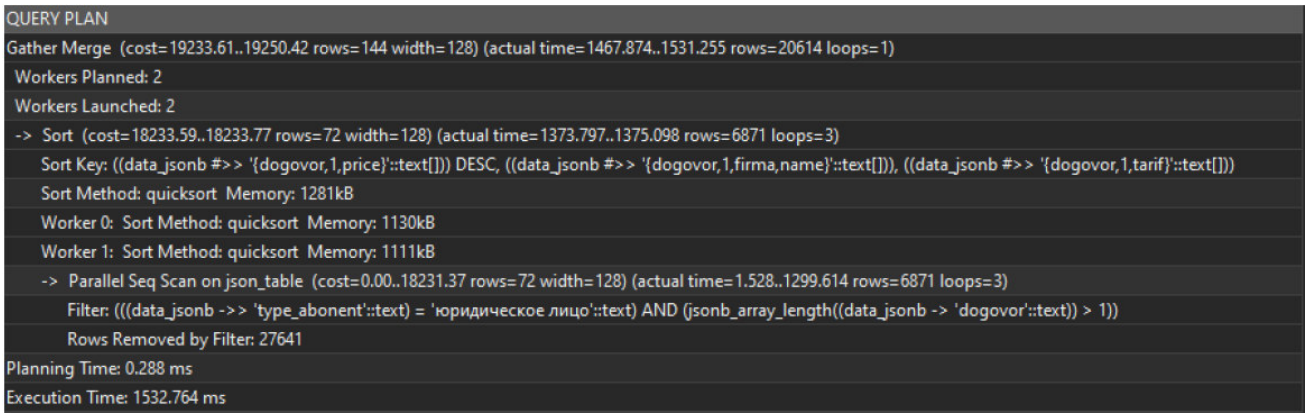
– оценка затратности операции (cost) уменьшилась значительно, было – 18842.87, стало – 418.67;

– по затраченному времени (Execution Time) на запрос показатель уменьшился, было – 4401.948 ms, стало – 3018.863 ms;

– последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan (используется индекс json_table_data_jsonb_idx для определения нужных нам записей) а затем PostgreSQL лезет в саму таблицу: (Bitmap Heap Scan), чтобы убедиться, что эти записи на самом деле существуют.)

Исходный запрос:

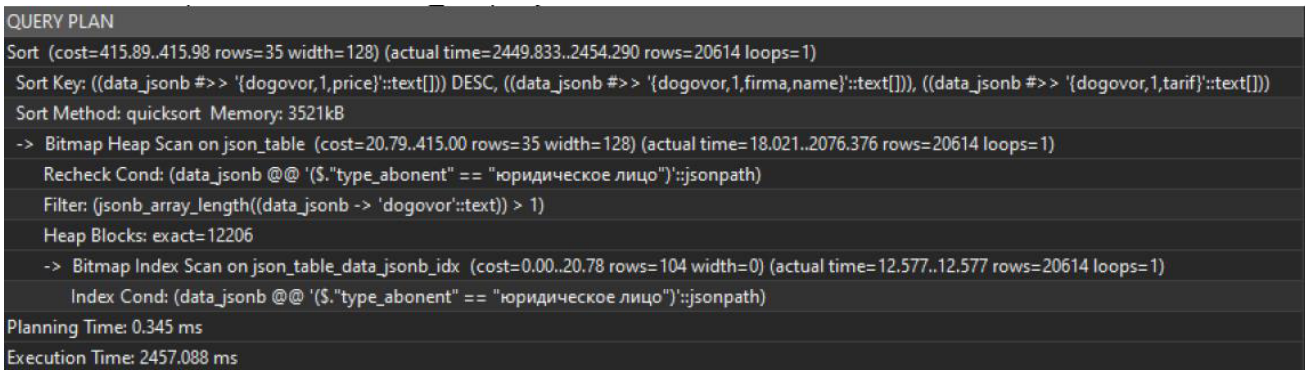
```
EXPLAIN (ANALYZE)
   SELECT data_jsonb ->>'name' as "name",
          data_jsonb #>>'{dogovor,1,firma,name}' as name_company,
          data_jsonb #>>'{dogovor,1,tarif}' as tarif,
          data_jsonb #>>'{dogovor,1,price}' as price
   FROM json_table
   WHERE data_jsonb->>'type_abonent'='юридическое лицо'
         AND jsonb_array_length(data_jsonb->'dogovor')>1
   ORDER BY price DESC, name_company, tarif;
```



```
QUERY PLAN
Gather Merge  (cost=19233.61..19250.42 rows=144 width=128) (actual time=1467.874..1531.255 rows=20614 loops=1)
 Workers Planned: 2
 Workers Launched: 2
 ->  Sort  (cost=18233.59..18233.77 rows=72 width=128) (actual time=1373.797..1375.098 rows=6871 loops=3)
     Sort Key: ((data_jsonb #>> '{dogovor,1,price}'::text[])) DESC, ((data_jsonb #>> '{dogovor,1,firma,name}'::text[])), ((data_jsonb #>> '{dogovor,1,tarif}'::text[]))
     Sort Method: quicksort  Memory: 1281kB
     Worker 0:  Sort Method: quicksort  Memory: 1130kB
     Worker 1:  Sort Method: quicksort  Memory: 1111kB
     ->  Parallel Seq Scan on json_table  (cost=0.00..18231.37 rows=72 width=128) (actual time=1.528..1299.614 rows=6871 loops=3)
         Filter: (((data_jsonb ->> 'type_abonent'::text) = 'юридическое лицо'::text) AND (jsonb_array_length((data_jsonb -> 'dogovor'::text)) > 1))
         Rows Removed by Filter: 27641
Planning Time: 0.288 ms
Execution Time: 1532.764 ms
```

Оптимизированный запрос:

```
EXPLAIN (ANALYZE)
   SELECT data_jsonb ->>'name' as "name",
          data_jsonb #>>'{dogovor,1,firma,name}' as name_company,
          data_jsonb #>>'{dogovor,1,tarif}' as tarif,
          data_jsonb #>>'{dogovor,1,price}' as price
   FROM json_table
   WHERE data_jsonb @@ '$.type_abonent == "юридическое лицо"'
         AND jsonb_array_length(data_jsonb->'dogovor')>1
   ORDER BY price DESC, name_company, tarif;
```



```
QUERY PLAN
Sort  (cost=415.89..415.98 rows=35 width=128) (actual time=2449.833..2454.290 rows=20614 loops=1)
 Sort Key: ((data_jsonb #>> '{dogovor,1,price}'::text[])) DESC, ((data_jsonb #>> '{dogovor,1,firma,name}'::text[])), ((data_jsonb #>> '{dogovor,1,tarif}'::text[]))
 Sort Method: quicksort  Memory: 3521kB
 ->  Bitmap Heap Scan on json_table  (cost=20.79..415.00 rows=35 width=128) (actual time=18.021..2076.376 rows=20614 loops=1)
     Recheck Cond: (data_jsonb @@ '($."type_abonent" == "юридическое лицо")'::jsonpath)
     Filter: (jsonb_array_length((data_jsonb -> 'dogovor'::text)) > 1)
     Heap Blocks: exact=12206
     ->  Bitmap Index Scan on json_table_data_jsonb_idx  (cost=0.00..20.78 rows=104 width=0) (actual time=12.577..12.577 rows=20614 loops=1)
         Index Cond: (data_jsonb @@ '($."type_abonent" == "юридическое лицо")'::jsonpath)
Planning Time: 0.345 ms
Execution Time: 2457.088 ms
```

По результатам видим, что:

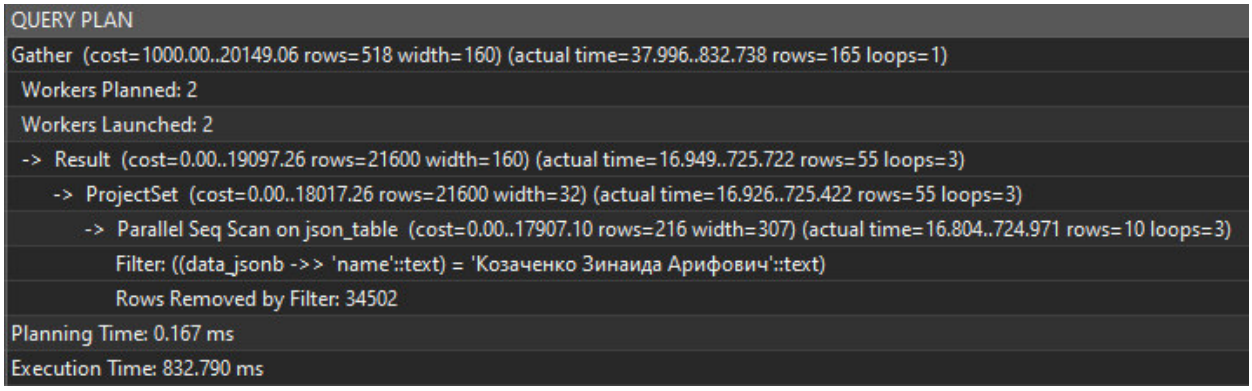
– оценка затратности операции (cost) уменьшилась значительно, было – 19250.42, стало – 415.98;

– по затраченному времени (Execution Time) на запрос показатель увеличился, было – 1532.764 ms, стало – 2457.088 ms;

– последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan (используется индекс json_table_data_jsonb_idx для определения нужных нам записей).

Исходный запрос:

```
EXPLAIN (ANALYZE)
SELECT (jsonb_array_elements(data_jsonb->'dogovor'))->'firma'->>'name' as "name",
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'number_dogovora' as num,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'tarif' as tarif,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'data' as "data",
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'dop_usluga' as dop
FROM json_table
WHERE data_jsonb ->>'name'='Козаченко Зинаида Арифович';
```



Оптимизированный запрос:

```
EXPLAIN (ANALYZE)
SELECT (jsonb_array_elements(data_jsonb->'dogovor'))->'firma'->>'name' as "name",
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'number_dogovora' as num,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'tarif' as tarif,
       (jsonb_array_elements(data_jsonb->'dogovor'))->>'data' as "data",
      (jsonb_array_elements(data_jsonb->'dogovor'))->>'dop_usluga' as dop
FROM json_table
WHERE data_jsonb @> '{"name": "Козаченко Зинаида Арифович"}';
```

```
QUERY PLAN
Result  (cost=20.80..986.92 rows=10400 width=160) (actual time=0.226..3.551 rows=165 loops=1)
 ->  ProjectSet  (cost=20.80..466.92 rows=10400 width=32) (actual time=0.217..2.526 rows=165 loops=1)
    ->  Bitmap Heap Scan on json_table  (cost=20.80..413.88 rows=104 width=307) (actual time=0.174..1.477 rows=29 loops=1)
        Recheck Cond: (data_jsonb @> '{"name": "Козаченко Зинаида Арифович"}'::jsonb)
        Heap Blocks: exact=29
        ->  Bitmap Index Scan on json_table_data_jsonb_idx  (cost=0.00..20.78 rows=104 width=0) (actual time=0.106..0.106 rows=29 loops=1)
            Index Cond: (data_jsonb @> '{"name": "Козаченко Зинаида Арифович"}'::jsonb)
Planning Time: 0.357 ms
Execution Time: 3.675 ms
```

По результатам видим, что:

– оценка затратности операции (cost) уменьшилась значительно, было – 20149.06, стало – 986.92;

– по затраченному времени (Execution Time) на запрос показатель уменьшился значительно, было – 832.790 ms, стало – 3.675 ms;

– последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan (используется индекс json_table_data_jsonb_idx для определения нужных нам записей).

Исходный запрос:

```
EXPLAIN (ANALYZE) SELECT data_jsonb->>'name',
                         data_jsonb->>'address',
                         data_jsonb->>'phone'
       FROM json_table
       WHERE data_jsonb @@ '$.dogovor[*].firma.name == "Trinity"';
```

```
QUERY PLAN
Bitmap Heap Scan on json_table  (cost=20.80..414.66 rows=104 width=96) (actual time=8.208..2204.111 rows=34060 loops=1)
  Recheck Cond: (data_jsonb @@ '($."dogovor"[*]."firma"."name" == "Trinity")'::jsonpath)
  Heap Blocks: exact=13208
  ->  Bitmap Index Scan on json_table_data_jsonb_idx  (cost=0.00..20.78 rows=104 width=0) (actual time=6.189..6.189 rows=34060 loops=1)
        Index Cond: (data_jsonb @@ '($."dogovor"[*]."firma"."name" == "Trinity")'::jsonpath)
Planning Time: 0.553 ms
Execution Time: 2207.697 ms
```

Данный запрос можно оставить в прежнем виде т.к. по результатам видим, что используется сканирование по индексу (Bitmap Index Scan on json_table_data_jsonb_idx), оценка затратности операции (cost) составляет 414.66, что является небольшим по сравнению с предыдущими запросами. По затраченному времени (Execution Time) на запрос показатель равен 2207.697 ms, что также является не большим.

Исходный запрос:

```
EXPLAIN (ANALYZE) SELECT data_jsonb ->>'name',
                         jsonb_path_query(data_jsonb,'$.dogovor.size()')
FROM json_table
WHERE jsonb_path_exists(data_jsonb,'$.dogovor[*] ? (@.firma.name=="DTK")
                                                 ? (@.tarif =="ПРЕМИУМ HD")');
```

```
QUERY PLAN
Gather  (cost=1000.00..94294.25 rows=34512 width=64) (actual time=1.184..1080.607 rows=2636 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  ProjectSet  (cost=0.00..89843.05 rows=14380000 width=64) (actual time=0.429..978.150 rows=879 loops=3)
        ->  Parallel Seq Scan on json_table  (cost=0.00..17799.25 rows=14380 width=307) (actual time=0.393..929.569 rows=879 loops=3)
              Filter: jsonb_path_exists(data_jsonb, '$."dogovor"[*]?(@."firma"."name" == "DTK")?(@."tarif" == "ПРЕМИУМ HD")'::jsonpath, '{}'::jsonb, false)
              Rows Removed by Filter: 33633
Planning Time: 0.107 ms
Execution Time: 1081.004 ms
```

Оптимизированный запрос:

```
EXPLAIN (ANALYZE) SELECT data_jsonb ->>'name',
                         jsonb_path_query(data_jsonb,'$.dogovor.size()')
FROM json_table
WHERE data_jsonb @@ 'exists($.dogovor[*] ? (@.firma.name=="DTK")
                                        ? (@.tarif=="ПРЕМИУМ HD"))';
```

```
QUERY PLAN
ProjectSet  (cost=36.80..950.92 rows=104000 width=64) (actual time=7.307..438.330 rows=2636 loops=1)
  ->  Bitmap Heap Scan on json_table  (cost=36.80..429.88 rows=104 width=307) (actual time=7.293..332.010 rows=2636 loops=1)
        Recheck Cond: (data_jsonb @@ 'exists ($."dogovor"[*]?(@."firma"."name" == "DTK")?(@."tarif" == "ПРЕМИУМ HD"))'::jsonpath)
        Rows Removed by Index Recheck: 8524
        Heap Blocks: exact=6586
        ->  Bitmap Index Scan on json_table_data_jsonb_idx  (cost=0.00..36.78 rows=104 width=0) (actual time=6.011..6.011 rows=11160 loops=1)
              Index Cond: (data_jsonb @@ 'exists ($."dogovor"[*]?(@."firma"."name" == "DTK")?(@."tarif" == "ПРЕМИУМ HD"))'::jsonpath)
Planning Time: 0.115 ms
Execution Time: 438.933 ms
```

По результатам видим, что:

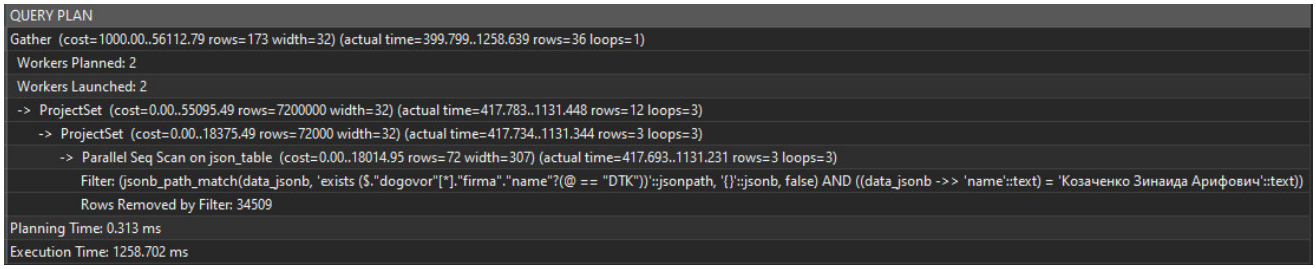
– оценка затратности операции (cost) уменьшилась значительно, было – 94294.25, стало – 950.92;

– по затраченному времени (Execution Time) на запрос показатель уменьшился значительно, было – 1081.004 ms, стало – 438.933 ms;

– последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan (используется индекс json_table_data_jsonb_idx для определения нужных нам записей).

Исходный запрос:

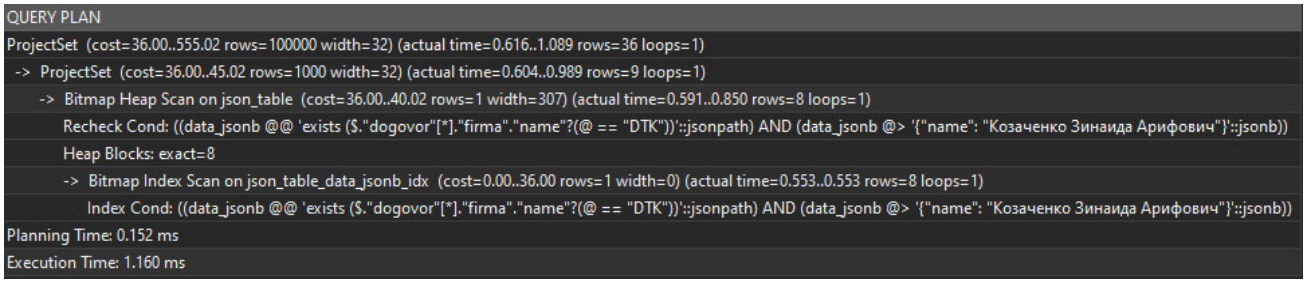
```
EXPLAIN (ANALYZE) SELECT
jsonb_each_text(jsonb_path_query(data_jsonb,'$.dogovor[*].firma?(@.name=="DTK")'))
FROM json_table
WHERE jsonb_path_match(data_jsonb, 'exists($.dogovor[*].firma.name?(@=="DTK"))')
      AND data_jsonb ->>'name'='Козаченко Зинаида Арифович';
```



```
QUERY PLAN
Gather  (cost=1000.00..56112.79 rows=173 width=32) (actual time=399.799..1258.639 rows=36 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  ProjectSet  (cost=0.00..55095.49 rows=7200000 width=32) (actual time=417.783..1131.448 rows=12 loops=3)
        ->  ProjectSet  (cost=0.00..18375.49 rows=72000 width=32) (actual time=417.734..1131.344 rows=3 loops=3)
              ->  Parallel Seq Scan on json_table  (cost=0.00..18014.95 rows=72 width=307) (actual time=417.693..1131.231 rows=3 loops=3)
                    Filter: (jsonb_path_match(data_jsonb, 'exists ($."dogovor"[*]."firma"."name"?(@ == "DTK"))'::jsonpath, '{}'::jsonb, false) AND ((data_jsonb ->> 'name'::text) = 'Козаченко Зинаида Арифович'::text))
                    Rows Removed by Filter: 34509
Planning Time: 0.313 ms
Execution Time: 1258.702 ms
```

Оптимизированный запрос:

```
EXPLAIN (ANALYZE) SELECT
jsonb_each_text(jsonb_path_query(data_jsonb,'$.dogovor[*].firma?(@.name=="DTK")'))
FROM json_table
WHERE data_jsonb @@ 'exists($.dogovor[*].firma.name ? (@ == "DTK"))' AND
      data_jsonb @>'{"name":"Козаченко Зинаида Арифович"}';
```



```
QUERY PLAN
ProjectSet  (cost=36.00..555.02 rows=100000 width=32) (actual time=0.616..1.089 rows=36 loops=1)
  ->  ProjectSet  (cost=36.00..45.02 rows=1000 width=32) (actual time=0.604..0.989 rows=9 loops=1)
        ->  Bitmap Heap Scan on json_table  (cost=36.00..40.02 rows=1 width=307) (actual time=0.591..0.850 rows=8 loops=1)
              Recheck Cond: ((data_jsonb @@ 'exists ($."dogovor"[*]."firma"."name"?(@ == "DTK"))'::jsonpath) AND (data_jsonb @> '{"name": "Козаченко Зинаида Арифович"}'::jsonb))
              Heap Blocks: exact=8
              ->  Bitmap Index Scan on json_table_data_jsonb_idx  (cost=0.00..36.00 rows=1 width=0) (actual time=0.553..0.553 rows=8 loops=1)
                    Index Cond: ((data_jsonb @@ 'exists ($."dogovor"[*]."firma"."name"?(@ == "DTK"))'::jsonpath) AND (data_jsonb @> '{"name": "Козаченко Зинаида Арифович"}'::jsonb))
Planning Time: 0.152 ms
Execution Time: 1.160 ms
```

По результатам видим, что:

– оценка затратности операции (cost) уменьшилась значительно, было – 56112.79, стало – 555.02;

– по затраченному времени (Execution Time) на запрос показатель уменьшился значительно, было – 1258.702 ms, стало – 1.160 ms;

– последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan (используется индекс json_table_data_jsonb_idx для определения нужных нам записей).

Задание к лабораторной работе

Для предметной области, описанной в заданном варианте для курсовой работы, выполнить требования, приведенные ниже.

Требования к выполняемой работе:

– Необходимо продумать и написать пример структуры JSON исходя из задания;

– Создать таблицу для хранения данных формата JSON;

– Заполнить таблицу большим количеством псевдо данными (не менее 100000). Глубина вложенности JSON данных не менее 4;

– Выборка из БД:

– Создать запросы, с выводом конкретных полей и их значений, а также объектов или массивов JSON документа, с одним ил несколькими условиями (не менее 3 штук);

– Выборки (запросы) с использованием jsonpath (не менее 3 штук);

– Выборки (запросы) с использованием функций обработки JSON (не менее 3 штук)

– Обновление/добавление/удаление данных в JSON документе (не менее 6 штук);

– Удаление (не менее 2 штук);

– Создать индекс (не менее 3 штук, 1 по полю в котором хранится JSON документ, и не менее 2 по определённым элементам ключей);

– Провести анализ созданных запросов (с помощью EXPLAIN и ANALYZE);

– Исходя из анализа созданных запросов оптимизировать их так чтобы они пользовались созданными индексами и при этом рассчитанная стоимость запуска и общая стоимость каждого узла плана не увеличилась (не менее 5 запросов оптимизировать).

Требования к отчету по лабораторной работе:

1. Титульный лист;
2. Тема лабораторной работы;
3. Задание на лабораторную работу;
4. Пример структуры JSON;
5. Примеры запросов из БД;
6. Код создания индексов;
7. Анализ и оптимизация запросов.

ЛАБОРАТОРНАЯ РАБОТА 8

Секционирование баз данных (партиционирование)

Цель работы: Изучение построения секционированных таблиц и приобретение навыков построения на их основе баз данных в PostgreSQL.

Методические указания к выполнению работы

Из задания к курсовому проекту возьмем описание одной из таблиц – «… экологических катастрофах (предприятие, тип катастрофы, дата, количество пострадавших людей, последствия, сведения об участии ЭС в устранении ЭК …».

На основе этого описания создадим таблицу eco_catastrophy как секционированную таблицу с предложением PARTITION BY, указав метод разбиения (в нашем случае RANGE) и список столбцов, которые будут образовывать ключ разбиения (в нашем случае это один столбец – data_catastrophe).

Создание партиции по диапазону:

```
CREATE TABLE eco_catastrophy (
      "id" SERIAL NOT NULL,
       id_company int4 NOT NULL,
       id_type_catastrophe int4 NOT NULL,
       data_catastrophe TIMESTAMP NOT NULL,
       number_victims int4 NOT NULL,
       effects TEXT,
       detriment DECIMAL,
       id_department int4[]
) PARTITION BY RANGE (data_catastrophe);
```

Создание секций:

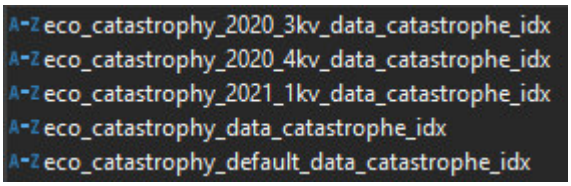
В определении каждой секции должны задаваться границы, соответствующие методу и ключу разбиения родительской таблицы. Секции, создаваемые таким образом, во всех отношениях являются обычными таблицами PostgreSQL. Указание границ, при котором множество значений новой секции пересекается со множеством значений в одной или нескольких существующих секциях, будет ошибочным. В нашем примере каждая секция должна содержать данные за один квартал.

```
CREATE TABLE eco_catastrophy_2020_3kv PARTITION OF eco_catastrophy
    FOR VALUES FROM ('2020-07-01') TO ('2020-10-01');
CREATE TABLE eco_catastrophy_2020_4kv PARTITION OF eco_catastrophy
    FOR VALUES FROM ('2020-10-01') TO ('2021-01-01');
CREATE TABLE eco_catastrophy_2021_1kv PARTITION OF eco_catastrophy
    FOR VALUES FROM ('2021-01-01') TO ('2021-04-01');
CREATE TABLE eco_catastrophy_default PARTITION OF eco_catastrophy DEFAULT;
```

Создание индекса:

Создадим в секционируемой таблице индекс по ключевому столбцу – *data_catastrophe*. При этом автоматически будет создан соответствующий индекс в каждой секции и все секции, которые вы будете создавать или присоединять позднее, тоже будут содержать такой индекс. Индекс по ключу, строго говоря, создавать не обязательно, но в большинстве случаев он будет полезен

```
CREATE INDEX ON eco_catastrophy (data_catastrophe);
```



Добавление данных:

```
INSERT INTO eco_catastrophy (id_company, id_type_catastrophe,
data_catastrophe, number_victims, effects, detriment, id_department)
    SELECT
        trunc(random() * 10),
        trunc(random() * 1000),
            now() - '3 month'::interval * random(),
        trunc(random() * 10000),
            md5(random()::text),
        (random() * 100000)::decimal,
        mas_if((trunc(random() * 10))::int4)
    FROM
        generate_series(1, 100000);
```

Запускаем данную команду несколько раз, с разными параметрами интервала дат (*now() - '8 month'::interval * random()*, *now() - '6 month'::interval * random()*, *now() - '4 month'::interval * random()*, *now() - '2 month'::interval * random()*, *now() + '2 month'::interval * random()*) и получаем 500 000 рандомных записей. Функция *mas_if(in_cnt)* – это пользовательская функция, которая на вход принимает размер массива (*in_cnt*), целое число, а на выходе возвращает массив рандомных целых чисел [1].

Результат добавления данных:

```
SELECT
  (SELECT count(*) FROM eco_catastrophy_2020_3kv) as eco_catastrophy_2020_3kv,
  (SELECT count(*) FROM eco_catastrophy_2020_4kv) as eco_catastrophy_2020_4kv,
  (SELECT count(*) FROM eco_catastrophy_2021_1kv) as eco_catastrophy_2021_1kv,
  (SELECT count(*) FROM eco_catastrophy_default) as eco_catastrophy_default,
  (SELECT count(*) FROM ONLY eco_catastrophy) as eco_catastrophy;
```

eco_catastrophy_2020_3kv	eco_catastrophy_2020_4kv	eco_catastrophy_2021_1kv	eco_catastrophy_default	eco_catastrophy
50326	134046	260650	54978	0

Создаем новую таблицу вне структуры секций и загружаем в нее данные из секции eco_catastrophy_default:

```
CREATE TABLE eco_catastrophy_2021_2kv
      (LIKE eco_catastrophy INCLUDING DEFAULTS INCLUDING CONSTRAINTS);
ALTER TABLE eco_catastrophy_2021_2kv ADD CONSTRAINT catastrophy_2021_2kv
      CHECK (data_catastrophe >= DATE '2021-04-01' AND
            data_catastrophe < DATE '2021-07-01');
INSERT INTO eco_catastrophy_2021_2kv SELECT * FROM eco_catastrophy_default;
```

С помощью предложение LIKE определяем таблицу, из которой в новую таблицу будут автоматически скопированы все имена столбцов, их типы данных и их ограничения на NULL. Дополняем командами что и как будем копировать: INCLUDING DEFAULTS – скопировать выражения значений по умолчанию в определениях копируемых столбцов, без этого указания выражения по умолчанию не копируются, и INCLUDING CONSTRAINTS – скопировать ограничения-проверки.

Создаем ограничение CHECK в присоединяемой таблице, соответствующее ожидаемому ограничению секции. Благодаря этому система сможет обойтись без сканирования, необходимого для проверки неявного ограничения секции.

Копируем все данные, которые хранятся в дефолтной секции в новую таблицу, которая пока еще не является секцией нашей партиции – *eco_catastrophy*.

```
INSERT INTO eco_catastrophy_2021_2kv SELECT * FROM eco_catastrophy_default
> Affected rows: 54978
> Time: 0,491s
```

Удаляем все данные из дефолтной секции:

```
DELETE FROM eco_catastrophy_default;
```

```
DELETE FROM eco_catastrophy_default
> Affected rows: 54978
> Time: 0,041s
```

Делаем таблицу eco_catastrophy_2021_2kv секцией:

```
ALTER TABLE eco_catastrophy ATTACH PARTITION eco_catastrophy_2021_2kv
                        FOR VALUES FROM ('2021-04-01') TO ('2021-07-01');
```

```
QUERY PLAN
Merge Append  (cost=0.45..7756.52 rows=55568 width=110) (actual time=0.012..318.898 rows=54978 loops=1)
  Sort Key: eco_catastrophy_2021_2kv.data_catastrophe
  ->  Index Scan using eco_catastrophy_2021_2kv_data_catastrophe_idx on eco_catastrophy_2021_2kv  (cost=0.29..6270.35 rows=54978 width=110) (actu
        Index Cond: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
  ->  Index Scan using eco_catastrophy_default_data_catastrophe_idx on eco_catastrophy_default  (cost=0.15..930.47 rows=590 width=109) (actual time
        Index Cond: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
Planning Time: 10.213 ms
Execution Time: 320.335 ms
```

Исходя из построенного плана видим, что поиск идет по новой добавленной секции и когда мы делали операцию добавления новой секции, создался новый индекс (*eco_catastrophy_2021_2kv_data_catastrophe_idx*) по полю data_*catastrophe*.

Создание партиции по списку:

```
CREATE TABLE eco_catastrophy_list (
      "id" SERIAL NOT NULL,
       id_company int4 NOT NULL,
       id_type_catastrophe int4 NOT NULL,
       data_catastrophe TIMESTAMP NOT NULL,
       number_victims int4 NOT NULL,
       effects TEXT,
       detriment DECIMAL,
       id_department int4[]
) PARTITION BY LIST ("id_company");
```

Создание секций:

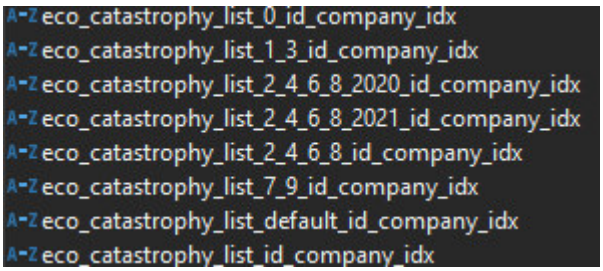
```
CREATE TABLE eco_catastrophy_list_0 PARTITION OF eco_catastrophy_list
    FOR VALUES IN (0);
CREATE TABLE eco_catastrophy_list_1_3 PARTITION OF eco_catastrophy_list
    FOR VALUES IN (1,3,5);
CREATE TABLE eco_catastrophy_list_7_9 PARTITION OF eco_catastrophy_list
    FOR VALUES IN (7,9);
CREATE  TABLE  eco_catastrophy_list_default  PARTITION  OF  eco_catastrophy_list
DEFAULT;
CREATE TABLE eco_catastrophy_list_2_4_6_8 PARTITION OF eco_catastrophy_list
    FOR VALUES IN (2,4,6,8)
    PARTITION BY RANGE (data_catastrophe);
```

Создание вложенных секций:

```
CREATE TABLE eco_catastrophy_list_2_4_6_8_2020 PARTITION OF
      eco_catastrophy_list_2_4_6_8 FOR VALUES FROM ('2020-01-01') TO ('2021-01-
01');
CREATE TABLE eco_catastrophy_list_2_4_6_8_2021 PARTITION OF
      eco_catastrophy_list_2_4_6_8  FOR VALUES FROM ('2021-01-01') TO ('2022-01-
01');
```

Создание индекса:

```
CREATE INDEX ON eco_catastrophy_list ("id_company");
```



Результат скопированных данных из eco_catastrophy:

```
SELECT tableoid::regclass AS partition, count(*) FROM eco_catastrophy_list
	GROUP BY tableoid;
```

partition	count
eco_catastrophy_list_0	50115
eco_catastrophy_list_1_3	150255
eco_catastrophy_list_7_9	99576
eco_catastrophy_list_2_4_6_8_2020	73562
eco_catastrophy_list_2_4_6_8_2021	126492

Создание партиции по хешу:

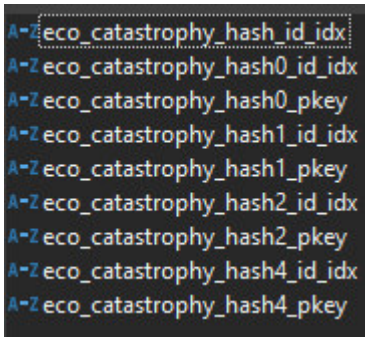
```
CREATE TABLE eco_catastrophy_hash (
	"id" SERIAL PRIMARY KEY NOT NULL,
	 id_company int4 NOT NULL,
	 id_type_catastrophe int4 NOT NULL,
	 data_catastrophe TIMESTAMP NOT NULL,
	 number_victims int4 NOT NULL,
	 effects TEXT,
	 detriment DECIMAL,
	 id_department int4[]
) PARTITION BY HASH ("id");
```

Создание секций:

```
CREATE TABLE eco_catastrophy_hash0 PARTITION OF eco_catastrophy_hash
	FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE eco_catastrophy_hash1 PARTITION OF eco_catastrophy_hash
	FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE eco_catastrophy_hash2 PARTITION OF eco_catastrophy_hash
	FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE eco_catastrophy_hash4 PARTITION OF eco_catastrophy_hash
	FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Создание индекса:

```
CREATE INDEX ON eco_catastrophy_hash ("id");
```



Заполнение с автоматической раскладкой по секциям:

```
INSERT INTO eco_catastrophy_hash
	SELECT * FROM eco_catastrophy ON CONFLICT ("id") DO NOTHING;
```

Распределение строк по секциям происходит равномерно:

```
SELECT tableoid::regclass AS partition, count(*) FROM eco_catastrophy_hash
    GROUP BY tableoid;
```

partition	count
eco_catastrophy_hash0	124817
eco_catastrophy_hash1	125337
eco_catastrophy_hash2	124916
eco_catastrophy_hash4	124930

Открепить секцию (убрать секцию из главной таблицы):

```
ALTER TABLE eco_catastrophy_hash DETACH PARTITION eco_catastrophy_hash4;
```

```
ALTER TABLE eco_catastrophy_hash DETACH PARTITION eco_catastrophy_hash4
> OK
> Time: 0,014s
```

Переименуем таблицу:

```
ALTER TABLE eco_catastrophy_hash4 RENAME TO eco_catastrophy_hash3;
```

```
ALTER TABLE eco_catastrophy_hash4 RENAME TO eco_catastrophy_hash3
> OK
> Time: 0,004s
```

Делаем таблицу eco_catastrophy_hash3 снова секцией:

```
ALTER TABLE eco_catastrophy_hash ATTACH PARTITION eco_catastrophy_hash3
                    FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Сравнение и анализ партиций и обычной таблицы:

Создадим обычную таблицу (*eco_catastrophy_test*) с такими же полями как у партиций и добавим индекс по *id* (*CREATE INDEX ON eco_catastrophy_test ("id")* и заполним данными скопировав из партиции *eco_catastrophy*).

```
EXPLAIN (ANALYZE) SELECT * FROM eco_catastrophy;
```

QUERY PLAN
Append (cost=0.00..17583.85 rows=500590 width=109) (actual time=0.172..109.407 rows=500000 loops=1)
-> Seq Scan on eco_catastrophy_2020_3kv (cost=0.00..2018.26 rows=50326 width=110) (actual time=0.171..15.838 rows=50326 loops=1)
-> Seq Scan on eco_catastrophy_2020_4kv (cost=0.00..4351.46 rows=134046 width=110) (actual time=0.324..29.372 rows=134046 loops=1)
-> Seq Scan on eco_catastrophy_2021_1kv (cost=0.00..7180.50 rows=260650 width=109) (actual time=0.058..34.598 rows=260650 loops=1)
-> Seq Scan on eco_catastrophy_2021_2kv (cost=0.00..1514.78 rows=54978 width=110) (actual time=0.043..11.417 rows=54978 loops=1)
-> Seq Scan on eco_catastrophy_default (cost=0.00..15.90 rows=590 width=109) (actual time=0.012..0.012 rows=0 loops=1)
Planning Time: 8.036 ms
Execution Time: 119.112 ms

```
EXPLAIN (ANALYZE) SELECT * FROM eco_catastrophy_list;
```

```
QUERY PLAN
Append  (cost=0.00..16295.25 rows=500550 width=110) (actual time=0.029..83.043 rows=500000 loops=1)
  ->  Seq Scan on eco_catastrophy_list_0  (cost=0.00..1380.15 rows=50115 width=109) (actual time=0.028..9.338 rows=50115 loops=1)
  ->  Seq Scan on eco_catastrophy_list_1_3  (cost=0.00..4141.55 rows=150255 width=110) (actual time=0.020..20.373 rows=150255 loops=1)
  ->  Seq Scan on eco_catastrophy_list_2_4_6_8_2020  (cost=0.00..2027.62 rows=73562 width=110) (actual time=0.088..11.328 rows=73562 loops=1)
  ->  Seq Scan on eco_catastrophy_list_2_4_6_8_2021  (cost=0.00..3484.92 rows=126492 width=110) (actual time=0.016..13.976 rows=126492 loops=1)
  ->  Seq Scan on eco_catastrophy_list_7_9  (cost=0.00..2742.76 rows=99576 width=109) (actual time=0.021..10.563 rows=99576 loops=1)
  ->  Seq Scan on eco_catastrophy_list_default  (cost=0.00..15.50 rows=550 width=120) (actual time=0.020..0.020 rows=0 loops=1)
Planning Time: 12.254 ms
Execution Time: 92.455 ms
```

```
EXPLAIN (ANALYZE) SELECT * FROM eco_catastrophy_hash;
```

```
QUERY PLAN
Append  (cost=0.00..16277.00 rows=500000 width=109) (actual time=0.039..111.655 rows=500000 loops=1)
  ->  Seq Scan on eco_catastrophy_hash0  (cost=0.00..3439.17 rows=124817 width=109) (actual time=0.039..24.985 rows=124817 loops=1)
  ->  Seq Scan on eco_catastrophy_hash1  (cost=0.00..3453.37 rows=125337 width=109) (actual time=0.036..23.587 rows=125337 loops=1)
  ->  Seq Scan on eco_catastrophy_hash2  (cost=0.00..3442.16 rows=124916 width=109) (actual time=0.030..21.559 rows=124916 loops=1)
  ->  Seq Scan on eco_catastrophy_hash4  (cost=0.00..3442.30 rows=124930 width=110) (actual time=0.034..19.950 rows=124930 loops=1)
Planning Time: 7.492 ms
Execution Time: 123.029 ms
```

```
EXPLAIN (ANALYZE) SELECT * FROM eco_catastrophy_test;
```

```
QUERY PLAN
Seq Scan on eco_catastrophy_test  (cost=0.00..13776.00 rows=500000 width=110) (actual time=0.291..237.331 rows=500000 loops=1)
Planning Time: 5.364 ms
Execution Time: 249.222 ms
```

Из результатов видим, что по затраченному времени на запрос (Execution Time) показатель почти у всех на одном уровне (не считая *eco_catastrophy_test*), быстрее справилась партиция разбитая по списку (*eco_catastrophy_list*) – 92.455 ms, а медленнее обычная таблица (*eco_catastrophy_test*) – 249.222 ms, что почти в 2 раза, хотя количество записей одинаково.

Общее количество строк (rows), полученных при последовательном сканировании (Seq Scan) у всех одинаково – 500000. У партиций данный параметр можно посмотреть по каждой секции.

Оценка затратности операции (cost) у партиций почти одинаковы (eco_catastrophy – 17583.85, *eco_catastrophy_list* – 16295.25, *eco_catastrophy_hash* – 16277.00) а у обычной таблицы (*eco_catastrophy_test*) – 13766.00.

Добавим к запросу условие на дату и сортировку. Сортировка, для партиции – *eco_catastrophy* будет по полю *data_catastrophe* (поле, по которому разбита партиция на секции), для *eco_catastrophy_list* будет по полю *id_company* (поле, по которому разбита партиция по секциям), для *eco_catastrophy_hash* будет по полю *id* (поле, по которому разбита партиция по секциям) и для *eco_catastrophy_test* будет по полю *id*.

```
EXPLAIN (ANALYZE)
      SELECT * FROM eco_catastrophy
      WHERE data_catastrophe>='2021-04-01' ORDER BY data_catastrophe;
```

```
QUERY PLAN
Merge Append  (cost=0.45..7756.52 rows=55568 width=110) (actual time=0.020..34.861 rows=54978 loops=1)
  Sort Key: eco_catastrophy_2021_2kv.data_catastrophe
  ->  Index Scan using eco_catastrophy_2021_2kv_data_catastrophe_idx on eco_catastrophy_2021_2kv  (cost=0.29..6270.35 rows=54978 width=110) (actu
        Index Cond: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
  ->  Index Scan using eco_catastrophy_default_data_catastrophe_idx on eco_catastrophy_default  (cost=0.15..930.47 rows=590 width=109) (actual time
        Index Cond: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
Planning Time: 7.547 ms
Execution Time: 36.036 ms
```

```
 EXPLAIN (ANALYZE)
      SELECT * FROM eco_catastrophy_list
      WHERE data_catastrophe>='2021-04-01' ORDER BY id_company;
```

```
QUERY PLAN
Gather Merge  (cost=13436.41..18859.93 rows=46484 width=110) (actual time=127.618..162.163 rows=54978 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=12436.39..12494.49 rows=23242 width=110) (actual time=80.298..82.489 rows=18326 loops=3)
        Sort Key: eco_catastrophy_list_1_3.id_company
        Sort Method: external merge  Disk: 2968kB
        Worker 0:  Sort Method: quicksort  Memory: 1400kB
        Worker 1:  Sort Method: external merge  Disk: 2832kB
        ->  Parallel Append  (cost=0.00..10750.83 rows=23242 width=110) (actual time=30.496..64.008 rows=18326 loops=3)
              ->  Parallel Seq Scan on eco_catastrophy_list_1_3  (cost=0.00..3743.82 rows=9959 width=110) (actual time=18.188..21.900 rows=5504 loops=3)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 44581
              ->  Parallel Seq Scan on eco_catastrophy_list_2_4_6_8_2021  (cost=0.00..3150.09 rows=12881 width=110) (actual time=22.815..29.020 rows=11050 loops=2)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 52196
              ->  Parallel Seq Scan on eco_catastrophy_list_7_9  (cost=0.00..2479.18 rows=6519 width=109) (actual time=36.511..42.518 rows=10801 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 88775
              ->  Parallel Seq Scan on eco_catastrophy_list_0  (cost=0.00..1247.49 rows=3346 width=109) (actual time=18.587..21.820 rows=5564 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 44551
              ->  Parallel Seq Scan on eco_catastrophy_list_default  (cost=0.00..14.04 rows=108 width=120) (actual time=0.001..0.001 rows=0 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
Planning Time: 5.126 ms
Execution Time: 164.815 ms
```

```
EXPLAIN (ANALYZE)
    SELECT * FROM eco_catastrophy_hash
    WHERE data_catastrophe>='2021-04-01' ORDER BY "id";
```

```
QUERY PLAN
Gather Merge  (cost=15211.58..20514.69 rows=45452 width=109) (actual time=166.775..182.536 rows=54978 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=14211.56..14268.38 rows=22726 width=109) (actual time=94.725..96.156 rows=18326 loops=3)
        Sort Key: eco_catastrophy_hash1.id
        Sort Method: external merge  Disk: 4040kB
        Worker 0:  Sort Method: quicksort  Memory: 3369kB
        Worker 1:  Sort Method: quicksort  Memory: 1431kB
        ->  Parallel Append  (cost=0.00..12567.10 rows=22726 width=109) (actual time=57.129..82.609 rows=18326 loops=3)
              ->  Parallel Seq Scan on eco_catastrophy_hash1  (cost=0.00..3121.60 rows=8274 width=109) (actual time=47.016..55.011 rows=13933 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 111404
              ->  Parallel Seq Scan on eco_catastrophy_hash4  (cost=0.00..3111.60 rows=8114 width=110) (actual time=16.578..18.486 rows=4568 loops=3)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 37075
              ->  Parallel Seq Scan on eco_catastrophy_hash2  (cost=0.00..3111.50 rows=7920 width=109) (actual time=41.328..49.729 rows=13610 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 111306
              ->  Parallel Seq Scan on eco_catastrophy_hash0  (cost=0.00..3108.77 rows=7776 width=109) (actual time=76.030..83.853 rows=13730 loops=1)
                    Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
                    Rows Removed by Filter: 111087
Planning Time: 7.186 ms
Execution Time: 184.856 ms
```

```
EXPLAIN (ANALYZE)
    SELECT * FROM eco_catastrophy_test
    WHERE data_catastrophe>='2021-04-01' ORDER BY "id";
```

```
QUERY PLAN
Gather Merge  (cost=14053.40..19440.74 rows=46174 width=110) (actual time=122.495..161.342 rows=54978 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=13053.37..13111.09 rows=23087 width=110) (actual time=72.899..76.275 rows=18326 loops=3)
        Sort Key: id
        Sort Method: external merge  Disk: 2144kB
        Worker 0:  Sort Method: external merge  Disk: 2144kB
        Worker 1:  Sort Method: external merge  Disk: 2224kB
        ->  Parallel Seq Scan on eco_catastrophy_test  (cost=0.00..11380.17 rows=23087 width=110) (actual time=42.982..55.853 rows=18326 loops=3)
              Filter: (data_catastrophe >= '2021-04-01 00:00:00'::timestamp without time zone)
              Rows Removed by Filter: 148341
Planning Time: 2.898 ms
Execution Time: 164.194 ms
```

Из результатов видно, что первая партиция (*eco_catastrophy*) разбитая по диапазонам дат справилась лучше всех – по затраченному времени (Execution Time) на запрос – 36.036 ms, оценка затратности операции (cost) – 7756.52. Это очевидно потом-что условие поиска и сортировка происходила по ключу разбиения партиции на секции, что привело к тому что поиск нужных записей проводился в 2-х секциях – *eco_catastrophy_20201_2kv* (это секция с данными за период от 01.04.2021 до 01.07.2021, как раз попадающая под условие поиска

data_catastrophe>='2021-04-01') и *eco_catastrophy_default* (это секция где хранятся данные которые не попали в остальные секции по разным причинам, но на данный момент она пуста). Также стоит отметить, что, последовательное сканирование (Seq Scan) сменилось на Index Scan — используется индекс *eco_catastrophy_20201_2kv_data_catastrophe_idx* (для секции *eco_catastrophy_20201_2kv* и *eco_catastrophy_default_data_catastrophe_idx* для секции *eco_catastrophy_default*) для условий *WHERE* и читает таблицу при отборе строк.

Вторая (*eco_catastrophy_list*) и третья (*eco_catastrophy_hash*) партиции справилась хуже, им пришлось сканировать все секции в поисках нужных записей, т.к. ключом разбиение у второй является поле *id_company*, а у третей поле *id*, а не поле дат (data_catastrophe) как у первой партиции. Но стоит отметить, что, у второй партиции (*eco_catastrophy_list*) подсекция *eco_catastrophy_list_2_4_6_8_2020* не была задействована в сканировании. Напомню, что мы секцию *eco_catastrophy_list_2_4_6_8* разбили на 2 подсекции – *eco_catastrophy_list_2_4_6_8_2020* (подсекция с данными у которых поле *data_catastrophe* лежит в пределах 2020 года) и *eco_catastrophy_list_2_4_6_8_2021* (подсекция с данными у которых поле *data_catastrophe* лежит в пределах 2021 года). Поэтому т.к. у нас условие найти данные, у которых *data_catastrophe>='2021-04-01'*, секция *eco_catastrophy_list_2_4_6_8_2020* с данными за 2020 год отбрасывается. Потом партиции сортируются: вторая (*eco_catastrophy_list*) по *id_company*, третья (*eco_catastrophy_hash*) по *id*. Используется последовательное сканирование секций, но в параллельном режиме (Parallel Seq Scan).

Обычная таблица справилась чуть лучше. По затраченному времени (Execution Time) на запрос показатель равен 164.194 ms, что наравне со второй партицией (*eco_catastrophy_list*) – 164.815 ms. Оценка затратности операции (cost) – 19440.74, что хуже, чем у первой (7756.52) и второй (18859.93) партиций, но лучше, чем у третей (20514.69) партиции.

```
EXPLAIN (ANALYZE)
	SELECT * FROM eco_catastrophy WHERE id_company=5 ORDER BY
data_catastrophe;
```

```
QUERY PLAN
Gather Merge  (cost=15838.53..20819.61 rows=42692 width=109) (actual time=141.728..172.134 rows=50327 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=14838.50..14891.87 rows=21346 width=109) (actual time=94.757..97.017 rows=16776 loops=3)
        Sort Key: eco_catastrophy_2021_1kv.data_catastrophe
        Sort Method: external merge  Disk: 2512kB
        Worker 0:  Sort Method: external merge  Disk: 2136kB
        Worker 1:  Sort Method: quicksort  Memory: 2651kB
        ->  Parallel Append  (cost=0.00..13303.55 rows=21346 width=109) (actual time=0.255..78.437 rows=16776 loops=3)
              ->  Parallel Seq Scan on eco_catastrophy_2021_1kv  (cost=0.00..5931.55 rows=11081 width=109) (actual time=0.337..28.370 rows=8748 loops=3)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 78135
              ->  Parallel Seq Scan on eco_catastrophy_2020_4kv  (cost=0.00..3996.63 rows=8211 width=110) (actual time=8.025..52.330 rows=6739 loops=2)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 60285
              ->  Parallel Seq Scan on eco_catastrophy_2020_3kv  (cost=0.00..1885.04 rows=2920 width=110) (actual time=6.885..27.945 rows=4992 loops=1)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 45334
              ->  Parallel Seq Scan on eco_catastrophy_2021_2kv  (cost=0.00..1369.25 rows=3326 width=110) (actual time=0.036..13.862 rows=5614 loops=1)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 49364
              ->  Parallel Seq Scan on eco_catastrophy_default  (cost=0.00..14.34 rows=35 width=109) (actual time=0.001..0.001 rows=0 loops=1)
                    Filter: (id_company = 5)
Planning Time: 3.480 ms
Execution Time: 175.555 ms
```

```
EXPLAIN (ANALYZE)
	SELECT * FROM eco_catastrophy_list WHERE id_company=5 ORDER BY
id_company;
```

```
QUERY PLAN
Bitmap Heap Scan on eco_catastrophy_list_1_3  (cost=915.18..4184.45 rows=50421 width=110) (actual time=23.025..68.221 rows=50327 loops=1)
  Recheck Cond: (id_company = 5)
  Heap Blocks: exact=2639
  ->  Bitmap Index Scan on eco_catastrophy_list_1_3_id_company_idx  (cost=0.00..902.58 rows=50421 width=0) (actual time=22.586..22.586 rows=50327 loops=1)
        Index Cond: (id_company = 5)
Planning Time: 0.236 ms
Execution Time: 69.485 ms
```

```
EXPLAIN (ANALYZE)
    SELECT * FROM eco_catastrophy_hash WHERE id_company=5 ORDER BY "id";
```

```
QUERY PLAN
Gather Merge  (cost=15065.59..19964.53 rows=41988 width=109) (actual time=151.616..169.143 rows=50327 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=14065.57..14118.05 rows=20994 width=109) (actual time=106.395..108.315 rows=16776 loops=3)
        Sort Key: eco_catastrophy_hash1.id
        Sort Method: external merge  Disk: 3184kB
        Worker 0:  Sort Method: quicksort  Memory: 2727kB
        Worker 1:  Sort Method: quicksort  Memory: 2813kB
        ->  Parallel Append  (cost=0.00..12558.44 rows=20994 width=109) (actual time=0.308..96.590 rows=16776 loops=3)
              ->  Parallel Seq Scan on eco_catastrophy_hash1  (cost=0.00..3121.60 rows=7395 width=109) (actual time=0.427..27.563 rows=4211 loops=3)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 37568
              ->  Parallel Seq Scan on eco_catastrophy_hash4  (cost=0.00..3111.60 rows=7309 width=110) (actual time=0.227..39.776 rows=6271 loops=2)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 56195
              ->  Parallel Seq Scan on eco_catastrophy_hash2  (cost=0.00..3111.50 rows=7595 width=109) (actual time=0.023..71.875 rows=12711 loops=1)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 112205
              ->  Parallel Seq Scan on eco_catastrophy_hash0  (cost=0.00..3108.77 rows=7340 width=109) (actual time=0.037..52.043 rows=12442 loops=1)
                    Filter: (id_company = 5)
                    Rows Removed by Filter: 112375
Planning Time: 0.202 ms
Execution Time: 171.902 ms
```

```
EXPLAIN (ANALYZE)
    SELECT * FROM eco_catastrophy_test WHERE id_company=5 ORDER BY "id";
```

```
QUERY PLAN
Gather Merge  (cost=13887.79..18788.13 rows=42000 width=110) (actual time=105.399..131.946 rows=50327 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  ->  Sort  (cost=12887.77..12940.27 rows=21000 width=110) (actual time=59.105..60.348 rows=16776 loops=3)
        Sort Key: id
        Sort Method: external merge  Disk: 3208kB
        Worker 0:  Sort Method: quicksort  Memory: 2488kB
        Worker 1:  Sort Method: quicksort  Memory: 3022kB
        ->  Parallel Seq Scan on eco_catastrophy_test  (cost=0.00..11380.17 rows=21000 width=110) (actual time=0.294..49.014 rows=16776 loops=3)
              Filter: (id_company = 5)
              Rows Removed by Filter: 149891
Planning Time: 0.080 ms
Execution Time: 135.120 ms
```

Поменяв условие поиска записей в *WHERE* (теперь ищем все записи у которых поле *id_company* равно пяти), видно что вторая партиция (*eco_catastrophy_list*) разбитая по списку справилось лучше всех, по затраченному времени (Execution Time) на запрос – 69.485 ms, оценка затратности операции (cost) – 4184.45. Это потому, что условие поиска и сортировка происходила по ключу разбиения её секций. Поэтому поиск происходит только в одной секции – *eco_catastrophy_list_1_3*, где хранятся записи с *id_company* равным 1,3 и 5 что

удовлетворяет нашему условию *WHERE id_company=5*. Также стоит отметить, что, последовательное сканирование (Seq Scan) сменилось на Bitmap Index Scan — используется индекс *eco_catastrophy_list_1_3_id_company_idx* для определения нужных нам записей, а затем PostgreSQL лезет в саму таблицу (Bitmap Heap Scan), чтобы убедиться, что эти записи на самом деле существуют.

Первая партиция (eco_*catastrophy*) которую разбивали по диапазонам дат и третья партиция (*eco_catastrophy_hash*) разбитая по хешу, справилась хуже, т.к. поиск проводился по всем секциям партиций. Результаты первой партиции (*eco_catastrophy*) - по затраченному времени (Execution Time) на запрос – 175.555 ms, оценка затратности операции (cost) – 20819.61. Результаты третьей партиции (*eco_catastrophy_hash*) - по затраченному времени (Execution Time) на запрос – 171.902 ms, оценка затратности операции (cost) – 19964.53. У обеих партиций используется последовательное сканирование секций, но в параллельном режиме (Parallel Seq Scan).

Показатели обычной таблицы по затраченному времени (Execution Time) на запрос – 135.120 ms, оценка затратности операции (cost) – 18788.13.

Добавим еще индексов (по ключевым полям) к партициям и к обычной таблице, и проверим на других запросах, как будет происходит их построение и какие из индексов будут участвовать в построении запроса, а какие нет.

```
CREATE INDEX ON eco_catastrophy ("id_company");
CREATE INDEX ON eco_catastrophy ("id");
CREATE INDEX ON eco_catastrophy_list ("data_catastrophe");
CREATE INDEX ON eco_catastrophy_list ("id");
CREATE INDEX ON eco_catastrophy_hash ("id_company");
CREATE INDEX ON eco_catastrophy_hash ("data_catastrophe");
CREATE INDEX ON eco_catastrophy_test ("id_company");
CREATE INDEX ON eco_catastrophy_test ("data_catastrophe");

EXPLAIN (ANALYZE) SELECT "id", id_company, data_catastrophe::date,
                         number_victims, effects, detriment
FROM eco_catastrophy WHERE data_catastrophe>='2021-02-25' AND
                           id_company IN(7,5) ORDER BY "id", id_company;
```

```
QUERY PLAN
Sort  (cost=9730.88..9795.66 rows=25913 width=61) (actual time=79.862..82.285 rows=25674 loops=1)
 Sort Key: eco_catastrophy_2021_1kv.id, eco_catastrophy_2021_1kv.id_company
 Sort Method: external merge  Disk: 1808kB
 ->  Append  (cost=928.21..7831.28 rows=25913 width=61) (actual time=3.977..64.346 rows=25674 loops=1)
    ->  Bitmap Heap Scan on eco_catastrophy_2021_1kv  (cost=928.21..6322.31 rows=14685 width=61) (actual time=3.976..52.169 rows=14615 loops=1
       Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
       Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
       Rows Removed by Filter: 37692
       Heap Blocks: exact=4574
       ->  Bitmap Index Scan on eco_catastrophy_2021_1kv_id_company_idx  (cost=0.00..924.53 rows=52226 width=0) (actual time=3.490..3.490 rows=
          Index Cond: (id_company = ANY ('{7,5}'::integer[]))
    ->  Bitmap Heap Scan on eco_catastrophy_2021_2kv  (cost=202.70..1362.14 rows=11111 width=61) (actual time=0.738..11.075 rows=11059 loops=1
       Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
       Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
       Heap Blocks: exact=965
       ->  Bitmap Index Scan on eco_catastrophy_2021_2kv_id_company_idx  (cost=0.00..199.92 rows=11111 width=0) (actual time=0.643..0.643 rows=
          Index Cond: (id_company = ANY ('{7,5}'::integer[]))
    ->  Bitmap Heap Scan on eco_catastrophy_default  (cost=5.21..17.26 rows=117 width=61) (actual time=0.006..0.006 rows=0 loops=1)
       Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
       Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
       ->  Bitmap Index Scan on eco_catastrophy_default_id_company_idx  (cost=0.00..5.18 rows=117 width=0) (actual time=0.005..0.005 rows=0 loop
          Index Cond: (id_company = ANY ('{7,5}'::integer[]))
Planning Time: 8.845 ms
Execution Time: 84.292 ms
```

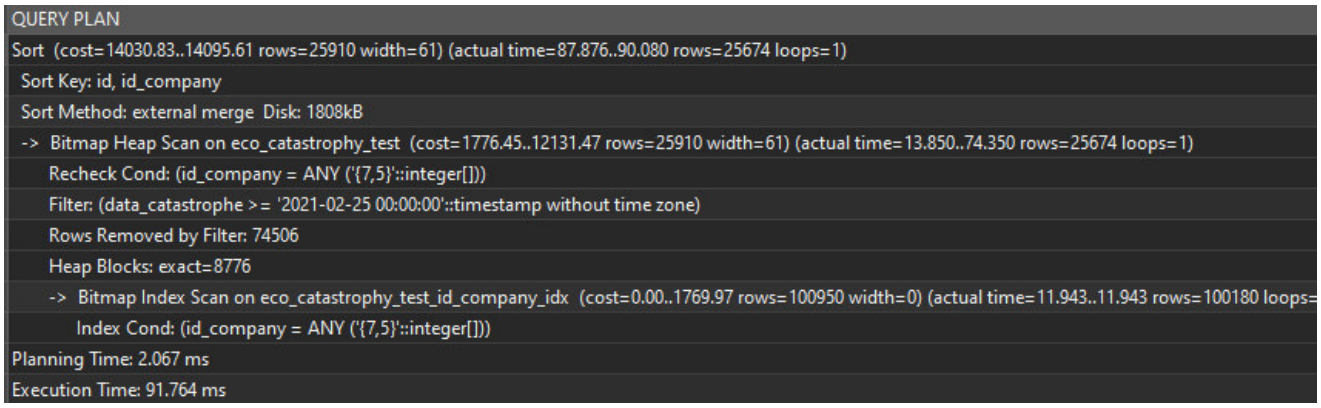
```
EXPLAIN (ANALYZE)SELECT "id", id_company, data_catastrophe::date,
                        number_victims, effects, detriment
FROM eco_catastrophy_list WHERE data_catastrophe>='2021-02-25' AND
                                id_company IN(7,5)
ORDER BY "id", id_company;
```

```
QUERY PLAN
Sort  (cost=8358.54..8423.16 rows=25849 width=61) (actual time=80.615..82.480 rows=25674 loops=1)
 Sort Key: eco_catastrophy_list_1_3.id, eco_catastrophy_list_1_3.id_company
 Sort Method: external merge  Disk: 1808kB
 ->  Append  (cost=0.42..6464.09 rows=25849 width=61) (actual time=0.070..62.989 rows=25674 loops=1)
    ->  Index Scan using eco_catastrophy_list_1_3_data_catastrophe_idx on eco_catastrophy_list_1_3  (cost=0.42..3799.74 rows=13075 width=61) (actual time=0.07
       Index Cond: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
       Filter: (id_company = ANY ('{7,5}'::integer[]))
       Rows Removed by Filter: 25377
    ->  Index Scan using eco_catastrophy_list_7_9_data_catastrophe_idx on eco_catastrophy_list_7_9  (cost=0.29..2535.11 rows=12774 width=61) (actual time=0.06
       Index Cond: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
       Filter: (id_company = ANY ('{7,5}'::integer[]))
       Rows Removed by Filter: 12684
Planning Time: 4.215 ms
Execution Time: 84.213 ms
```

```
EXPLAIN (ANALYZE) SELECT
 "id", id_company, data_catastrophe::date, number_victims, effects, detriment
FROM eco_catastrophy_hash WHERE data_catastrophe>='2021-02-25' AND
                                id_company IN(7,5) ORDER BY "id", id_company;
```

```
QUERY PLAN
Sort  (cost=14192.63..14257.34 rows=25884 width=61) (actual time=109.247..110.992 rows=25674 loops=1)
  Sort Key: eco_catastrophy_hash0.id, eco_catastrophy_hash0.id_company
  Sort Method: external merge  Disk: 1808kB
  ->  Append  (cost=456.98..12295.36 rows=25884 width=61) (actual time=10.362..95.598 rows=25674 loops=1)
        ->  Bitmap Heap Scan on eco_catastrophy_hash0  (cost=456.98..3045.18 rows=6458 width=61) (actual time=10.361..24.559 rows=6273 loops=1)
              Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
              Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
              Rows Removed by Filter: 18716
              Heap Blocks: exact=2191
              ->  Bitmap Index Scan on eco_catastrophy_hash0_id_company_idx  (cost=0.00..455.36 rows=25404 width=0) (actual time=1.626..1.626 rows=24989 loops=
                    Index Cond: (id_company = ANY ('{7,5}'::integer[]))
        ->  Bitmap Heap Scan on eco_catastrophy_hash1  (cost=446.70..3039.33 rows=6447 width=61) (actual time=9.694..25.013 rows=6421 loops=1)
              Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
              Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
              Rows Removed by Filter: 18622
              Heap Blocks: exact=2200
              ->  Bitmap Index Scan on eco_catastrophy_hash1_id_company_idx  (cost=0.00..445.08 rows=25101 width=0) (actual time=1.648..1.648 rows=25043 loops=
                    Index Cond: (id_company = ANY ('{7,5}'::integer[]))
                    Index Cond: (id_company = ANY ('{7,5}'::integer[]))
        ->  Bitmap Heap Scan on eco_catastrophy_hash2  (cost=457.99..3050.38 rows=6536 width=61) (actual time=11.589..25.965 rows=6447 loops=1)
              Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
              Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
              Rows Removed by Filter: 18676
              Heap Blocks: exact=2193
              ->  Bitmap Index Scan on eco_catastrophy_hash2_id_company_idx  (cost=0.00..456.35 rows=25537 width=0) (actual time=1.683..1.683 rows=25123 loops=
                    Index Cond: (id_company = ANY ('{7,5}'::integer[]))
        ->  Bitmap Heap Scan on eco_catastrophy_hash4  (cost=446.28..3031.04 rows=6443 width=61) (actual time=9.289..19.005 rows=6533 loops=1)
              Recheck Cond: (id_company = ANY ('{7,5}'::integer[]))
              Filter: (data_catastrophe >= '2021-02-25 00:00:00'::timestamp without time zone)
              Rows Removed by Filter: 18492
              Heap Blocks: exact=2193
              ->  Bitmap Index Scan on eco_catastrophy_hash4_id_company_idx  (cost=0.00..444.66 rows=25044 width=0) (actual time=1.659..1.659 rows=25025 loops=
                    Index Cond: (id_company = ANY ('{7,5}'::integer[]))
Planning Time: 8.479 ms
Execution Time: 112.640 ms
```

```
EXPLAIN (ANALYZE) SELECT "id", id_company, data_catastrophe::date,
                         number_victims, effects, detriment
FROM eco_catastrophy_test WHERE data_catastrophe>='2021-02-25' AND
                                id_company IN(7,5)
ORDER BY "id", id_company;
```



Из полученных результатов видим, что первая партиция (разбитая по диапазону) и вторая партиция (разбитая по спискам) справилась лучше всех.

Результаты первой партиции (eco_*catastrophy*) - по затраченному времени (Execution Time) на запрос – 84.292 ms, оценка затратности операции (cost) – 9795.66. Результаты второй партиции (*eco_catastrophy_list*) - по затраченному времени (Execution Time) на запрос – 84.213 ms, оценка затратности операции (cost) – 8423.16. В первой партиции поиск записей происходит с помощью Bitmap Index Scan — который использует индексы: *eco_catastrophy_2021_1kv_id_company_idx* для секции *eco_catastrophy_2021_1kv*, *eco_catastrophy_2021_2kv_id_company_idx* для секции *eco_catastrophy_2021_2kv* и *eco_catastrophy_default_id_company_idx* для секции *eco_catastrophy_default*, чтобы определить нужные нам записи, а затем PostgreSQL лезет в саму таблицу: (Bitmap Heap Scan), чтобы убедиться, что эти записи на самом деле существуют. Следует отметить, что это новый индекс, который мы создали для этой партиции по полю *id_company*. Во второй партиции поиск записей происходит с помощью Index Scan — который использует индексы: *eco_catastrophy_list_1_3_data_catastrophe_idx* для секции *eco_catastrophy_list_1_3*, *eco_catastrophy_list_7_9_data_catastrophe_idx* для секции *eco_catastrophy_list_7_9*, чтобы определить нужные нам записи при условий *WHERE*, читает таблицу при отборе строк. Следует отметить, что это новый индекс, который мы создали для этой партиции по полю *data_catastrophe*.

Третья партиция (*eco_catastrophy_hash*) справилась хуже всех. Результаты: по затраченному времени (Execution Time) на запрос – 112.640 ms, оценка затратности операции (cost) – 14257.34. Поиск записей происходит по всем секциям с помощью Bitmap Index Scan — который использует новый индекс, который мы создали для этой партиции по полю *id_company*.

Результаты таблицы *eco_catastrophy_test*: по затраченному времени (Execution Time) на запрос – 91.764 ms, оценка затратности операции (cost) – 14095.61. Обычная таблица по затраченному времени и по оценке затратности

операций уступает первой и второй партициям, но лучше, чем партиция разбитая по хешу но не на много. Также стоит отметить, что по сравнению с предыдущими запросами, последовательное сканирование (Seq Scan) или последовательное сканирование в параллельном режиме (Parallel Seq Scan) сменилось на Bitmap Index Scan — используется новый индекс *eco_catastrophy_test_id_company_idx* для определения нужных нам записей, а затем PostgreSQL лезет в саму таблицу (Bitmap Heap Scan), чтобы убедиться, что эти записи на самом деле существуют.

```
EXPLAIN (ANALYZE)
      SELECT id_company, max(detriment), min(detriment)
            FROM eco_catastrophy WHERE data_catastrophe>'2021-01-01'
                                    AND number_victims<1000
            GROUP BY id_company ORDER BY id_company;
```

```
QUERY PLAN
Finalize GroupAggregate  (cost=8842.81..8894.48 rows=200 width=68) (actual time=101.189..101.221 rows=10 loops=1)
 Group Key: eco_catastrophy_2021_1kv.id_company
 ->  Gather Merge  (cost=8842.81..8889.48 rows=400 width=68) (actual time=101.184..105.963 rows=30 loops=1)
     Workers Planned: 2
     Workers Launched: 2
     ->  Sort  (cost=7842.79..7843.29 rows=200 width=68) (actual time=48.472..48.473 rows=10 loops=3)
         Sort Key: eco_catastrophy_2021_1kv.id_company
         Sort Method: quicksort  Memory: 25kB
         Worker 0:  Sort Method: quicksort  Memory: 25kB
         Worker 1:  Sort Method: quicksort  Memory: 25kB
         ->  Partial HashAggregate  (cost=7833.14..7835.14 rows=200 width=68) (actual time=48.377..48.380 rows=10 loops=3)
             Group Key: eco_catastrophy_2021_1kv.id_company
             ->  Parallel Append  (cost=0.00..7734.28 rows=13182 width=16) (actual time=0.452..41.651 rows=10541 loops=3)
                 ->  Parallel Seq Scan on eco_catastrophy_2021_1kv  (cost=0.00..6203.06 rows=10867 width=16) (actual time=0.456..31.938 rows=8676 loo
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 78208
                 ->  Parallel Seq Scan on eco_catastrophy_2021_2kv  (cost=0.00..1450.10 rows=3267 width=16) (actual time=0.036..26.976 rows=5596 loop
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 49382
                 ->  Parallel Seq Scan on eco_catastrophy_default  (cost=0.00..15.21 rows=2 width=16) (actual time=0.005..0.005 rows=0 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
Planning Time: 2.064 ms
Execution Time: 106.090 ms
```

```
EXPLAIN (ANALYZE)
    SELECT id_company, max(detriment), min(detriment)
        FROM eco_catastrophy_list WHERE data_catastrophe>'2021-01-01'
                                    AND number_victims<1000
        GROUP BY id_company ORDER BY id_company;
```

```
QUERY PLAN
Finalize GroupAggregate  (cost=12436.33..12488.00 rows=200 width=68) (actual time=104.255..104.270 rows=10 loops=1)
 Group Key: eco_catastrophy_list_1_3.id_company
 ->  Gather Merge  (cost=12436.33..12483.00 rows=400 width=68) (actual time=104.251..125.693 rows=20 loops=1)
     Workers Planned: 2
     Workers Launched: 2
     ->  Sort  (cost=11436.31..11436.81 rows=200 width=68) (actual time=57.463..57.464 rows=7 loops=3)
         Sort Key: eco_catastrophy_list_1_3.id_company
         Sort Method: quicksort  Memory: 25kB
         Worker 0:  Sort Method: quicksort  Memory: 25kB
         Worker 1:  Sort Method: quicksort  Memory: 25kB
         ->  Partial HashAggregate  (cost=11426.67..11428.67 rows=200 width=68) (actual time=57.430..57.432 rows=7 loops=3)
             Group Key: eco_catastrophy_list_1_3.id_company
             ->  Parallel Append  (cost=0.00..11328.19 rows=13130 width=16) (actual time=10.046..51.919 rows=10541 loops=3)
                 ->  Parallel Seq Scan on eco_catastrophy_list_1_3  (cost=0.00..3964.78 rows=5675 width=16) (actual time=7.561..16.405 rows=3177 loops=3)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 46908
                 ->  Parallel Seq Scan on eco_catastrophy_list_2_4_6_8_2021  (cost=0.00..3336.11 rows=7271 width=16) (actual time=0.314..26.593 rows=6347 loops=
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 56899
                 ->  Parallel Seq Scan on eco_catastrophy_list_7_9  (cost=0.00..2625.61 rows=3706 width=16) (actual time=15.586..30.639 rows=6257 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 93319
                 ->  Parallel Seq Scan on eco_catastrophy_list_0  (cost=0.00..1321.19 rows=1849 width=16) (actual time=6.875..20.003 rows=3142 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 46973
                 ->  Parallel Seq Scan on eco_catastrophy_list_default  (cost=0.00..14.85 rows=36 width=36) (actual time=0.001..0.001 rows=0 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
Planning Time: 6.165 ms
Execution Time: 125.872 ms
```

```
EXPLAIN (ANALYZE)
      SELECT id_company, max(detriment), min(detriment)
            FROM eco_catastrophy_hash WHERE data_catastrophe>'2021-01-01'
                                        AND number_victims<1000
            GROUP BY id_company ORDER BY id_company;
```

```
QUERY PLAN
Finalize GroupAggregate  (cost=14363.94..14415.61 rows=200 width=68) (actual time=126.026..126.047 rows=10 loops=1)
 Group Key: eco_catastrophy_hash1.id_company
 ->  Gather Merge  (cost=14363.94..14410.61 rows=400 width=68) (actual time=126.017..131.798 rows=30 loops=1)
     Workers Planned: 2
     Workers Launched: 2
     ->  Sort  (cost=13363.92..13364.42 rows=200 width=68) (actual time=71.309..71.310 rows=10 loops=3)
         Sort Key: eco_catastrophy_hash1.id_company
         Sort Method: quicksort  Memory: 25kB
         Worker 0:  Sort Method: quicksort  Memory: 25kB
         Worker 1:  Sort Method: quicksort  Memory: 25kB
         ->  Partial HashAggregate  (cost=13354.28..13356.28 rows=200 width=68) (actual time=71.167..71.171 rows=10 loops=3)
             Group Key: eco_catastrophy_hash1.id_company
             ->  Parallel Append  (cost=0.00..13254.97 rows=13241 width=16) (actual time=19.216..65.534 rows=10541 loops=3)
                 ->  Parallel Seq Scan on eco_catastrophy_hash1  (cost=0.00..3305.91 rows=4665 width=16) (actual time=5.537..17.785 rows=2636 loops=3)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 39143
                 ->  Parallel Seq Scan on eco_catastrophy_hash4  (cost=0.00..3295.32 rows=4562 width=16) (actual time=7.529..20.051 rows=3925 loops=2)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 58541
                 ->  Parallel Seq Scan on eco_catastrophy_hash2  (cost=0.00..3295.20 rows=4755 width=16) (actual time=12.209..40.143 rows=7967 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 116949
                 ->  Parallel Seq Scan on eco_catastrophy_hash0  (cost=0.00..3292.33 rows=4712 width=16) (actual time=26.601..60.873 rows=7898 loops=1)
                     Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                     Rows Removed by Filter: 116919
Planning Time: 0.698 ms
Execution Time: 131.945 ms
```

```
EXPLAIN (ANALYZE)
      SELECT id_company, max(detriment), min(detriment)
            FROM eco_catastrophy_test WHERE data_catastrophe>'2021-01-01'
                                        AND number_victims<1000
            GROUP BY id_company ORDER BY id_company;
```

```
QUERY PLAN
Finalize GroupAggregate  (cost=13000.93..13003.52 rows=10 width=68) (actual time=105.145..105.164 rows=10 loops=1)
  Group Key: id_company
  ->  Gather Merge  (cost=13000.93..13003.27 rows=20 width=68) (actual time=105.137..118.582 rows=30 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        ->  Sort  (cost=12000.91..12000.94 rows=10 width=68) (actual time=56.376..56.377 rows=10 loops=3)
              Sort Key: id_company
              Sort Method: quicksort  Memory: 25kB
              Worker 0:  Sort Method: quicksort  Memory: 25kB
              Worker 1:  Sort Method: quicksort  Memory: 25kB
              ->  Partial HashAggregate  (cost=12000.65..12000.75 rows=10 width=68) (actual time=56.340..56.342 rows=10 loops=3)
                    Group Key: id_company
                    ->  Parallel Seq Scan on eco_catastrophy_test  (cost=0.00..11901.00 rows=13286 width=16) (actual time=0.394..51.561 rows=10541 loops=3)
                          Filter: ((data_catastrophe > '2021-01-01 00:00:00'::timestamp without time zone) AND (number_victims < 1000))
                          Rows Removed by Filter: 156126
Planning Time: 0.117 ms
Execution Time: 118.643 ms
```

Из полученных результатов видим что первая партиция (разбитая по диапазонам) производит поиск только в секциях где хранятся данные за 2021 год – это секция *eco_catastrophy_2021_1kv и секция eco_catastrophy_2021_2kv*, а также секция отвечающая за данные которые не попали ни в одну из секций – это секция *eco_catastrophy_default*. Результаты первой партиции (*eco_catastrophy*) - по затраченному времени (Execution Time) на запрос – 106.090 ms, оценка затратности операции (cost) – 8894.48. Следует отметить, что используется последовательное сканирование секций в параллельном режиме (Parallel Seq Scan).

Вторая партиция (разбитая по спискам) производит последовательное сканирование секций в параллельном режиме (Parallel Seq Scan) по всем секциям кроме одной – *eco_catastrophy_list_2_4_6_8_2020*. Напомню, что мы секцию *eco_catastrophy_list_2_4_6_8* разбили на 2 подсекции – *eco_catastrophy_list_2_4_6_8_2020* (подсекция с данными у которых поле *data_catastrophe* лежит в пределах 2020 года) и *eco_catastrophy_list_2_4_6_8_2021* (подсекция с данными у которых поле *data_catastrophe* лежит в пределах 2021 года). Поэтому т.к. у нас условие найти данные, у которых *data_catastrophe>'2021-01-01'*, секция *eco_catastrophy_list_2_4_6_8_2020* с данными за 2020 год отбрасывается, что нам на пользу. Результаты: по затраченному времени (Execution Time) на запрос – 125.872 ms, оценка затратности операции (cost) – 12488.00.

Третья партиция также производит последовательное сканирование секций в параллельном режиме (Parallel Seq Scan) по всем секциям. Результаты: по затраченному времени (Execution Time) на запрос – 131.945 ms, оценка затратности операции (cost) – 14415.61.

Результаты *eco_catastrophy_test*: по затраченному времени (Execution Time) на запрос – 118.643 ms, оценка затратности операции (cost) – 13003.52. Обычная таблица по затраченному времени и оценке затратности операций опять уступает партициям (первой (*eco_catastrophy*) – разбитая по диапазонам дат и второй (*eco_catastrophy_list*) – разбитая по спискам), но чуть лучше показывает результат по сравнению с третей партицией. Из этого следует что таблицы, которые хранят большое количество данных лучше преобразовывать в партиции.

Функция mas_if(in_cnt)

```
CREATE OR REPLACE FUNCTION "public"."mas_if"("in_cnt" int4)
  RETURNS "pg_catalog"."_int4" AS $BODY$
DECLARE
      arr_mas int[];
      tmp int;
BEGIN
      IF(in_cnt!=0 OR in_cnt IS NOT NULL)THEN
            FOR i IN 1..in_cnt LOOP
                  arr_mas = array_append(arr_mas,
((trunc(random()*100))::int4));
            END LOOP;
      END IF;
      RETURN arr_mas;
END;
$BODY$
  LANGUAGE plpgsql
```

Задание к лабораторной работе

Создать секционированную таблицу, метод разбиения секционированной таблицы указан в таблице 1 (Партиция №1). Создать не менее 4 секций, плюс секция по дефолту. Заполнить секционированную таблицу псевдоданными (не менее 500000 записей). Создать еще одну секционированную таблицу метод разбиения секционированной таблицы указан в таблице 1 (Партиция №2). Создать не менее 4 секций, плюс секция по дефолту. Скопировать данные из первой партиции. Создать простую таблицу, которая будет является копией секционированной таблицы. Скопировать данные из первой партиции. Создать индексы (один по ключу разбиения и еще несколько по ключевым полям) к секционированным таблицам и к обычной таблице. Провести сравнение и анализ партиций и обычной таблицы (с помощью EXPLAIN и ANALYZE).

Таблица 8.1. Варианты заданий

№	Партиция №1	Партиция №2	Дополнительно
1	по диапазону	по хешу	Присоединение новой таблицы к партиции №1
2	по списку	по хешу	Создание вложенных секций в партиции №1
3	по хешу	по диапазону	Открепление секции от партиции №1
4	по диапазону	по списку	Создание вложенных секций в партиции №1
5	по списку	по диапазону	Присоединение новой таблицы к партиции №2
6	по хешу	по списку	Создание вложенных секций в партиции №2
7	по диапазону	по списку	Присоединение новой таблицы к партиции №2
8	по списку	по диапазону	Создание вложенных секций в партиции №2
9	по хешу	по списку	Присоединение новой таблицы к партиции №2
10	по диапазону	по хешу	Открепление секции от партиции №2
11	по списку	по хешу	Открепление секции от партиции №1
12	по хешу	по диапазону	Присоединение новой таблицы к партиции №2
13	по диапазону	по списку	Открепление секции от партиции №2
14	по списку	по диапазону	Создание вложенных секций в партиции №2
15	по хешу	по диапазону	Открепление секции от партиции №1
16	по диапазону	по списку	Создание вложенных секций в партиции №1
17	по списку	по диапазону	Присоединение новой таблицы к партиции №1
18	по хешу	по списку	Создание вложенных секций в партиции №2
19	по диапазону	по хешу	Открепление секции от партиции №1
20	по списку	по хешу	Создание вложенных секций в партиции №1
21	по хешу	по списку	Присоединение новой таблицы к партиции №2
22	по диапазону	по списку	Создание вложенных секций в партиции №2
23	по списку	по диапазону	Открепление секции от партиции №1
24	по хешу	по диапазону	Открепление секции от партиции №2
25	по диапазону	по списку	Присоединение новой таблицы к партиции №1
26	по списку	по хешу	Открепление секции от партиции №2
27	по хешу	по диапазону	Присоединение новой таблицы к партиции №2
28	по диапазону	по списку	Создание вложенных секций в партиции №1
29	по списку	по хешу	Присоединение новой таблицы к партиции №1
30	по хешу	по диапазону	Открепление секции от партиции №2

Требования к отчету по лабораторной работе:

1. Титульный лист;
2. Тема лабораторной работы;
3. Задание на лабораторную работу;
4. Создание секционированных таблиц;
5. Создание секций;
6. Код создания индексов;
7. Добавление/удаление данных из секционированных таблиц;
8. Сравнение и анализ партиций и обычной таблицы.

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

к выполнению лабораторных работ по дисциплине
«Программирование систем с серверами баз данных»

Составители:
Щедрин Сергей Валерьевич – старший преподаватель кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДОННТУ»;
Бердюкова Светлана Сергеевна – ассистент кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДОННТУ».
Филипишин Дмитрий Александрович – ассистент кафедры программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДОННТУ»;

Ответственный за выпуск:
Зори Сергей Анатольевич —доктор технических наук, профессор, заведующий кафедрой программной инженерии им. Л.П. Фельдмана ФГБОУ ВО «ДонНТУ»,.